Lab 1: Getting started

The goals of this lab are:

1. Probability warmup
2. Familiarize ourselves with Jupyter notebooks
3. Use Python in the Jupyter Notebook
4. Perform mathmatical computations using the NumPy library
5. Import and manipulate data using the Pandas library
6. Plot data with Matplotlib and Seaborn
7. Practice exploring new datasets and visualizing them

Probability Practice (Show all work)

Question 1 (2 pts)

My neighbor has two children. Assuming that the gender of a child is like a coin flip, it is most likely, a priori, that my neighbor has one boy and one girl, with probability $1/2$. The other possibilities—two boys or two girls—have probabilities $1/4$ and $1/4$.

a. Suppose I ask him whether he has any boys, and he says yes. What is the probability that one child is a girl?

b. Suppose instead that I happen to see one of his children run by, and it is a boy. What is the probability that the other child is a girl?

Your solution:

a) Given the information I have, there are three viable outcomes - BB, BG, GB. Hence the probability that one child is a girl is 2/3

b) The probability that the second child is a girl is completely independent of what the sex of first chile is. Hence, the probability that the other child is a girl is 1/2

Question 2 (2 pts)

In 1995, they introduced blue M&M's. Before then, the color mix in a bag of plain M&M's was 30% Brown, 20% Yellow, 20% Red, 10% Green, 10% Orange, 10% Tan. Afterward it was 24% Blue, 20% Green, 16% Orange, 14% Yellow, 13% Red, 13% Brown.

Suppose a friend of mine has two bags of M&M's, and he tells me that one is from 1994 and one from 1996. He won't tell me which is which, but he gives me one M&M from

each bag. One is yellow and one is green. What is the probability that the yellow one came from the 1994 bag?

Your solution:

We need to find the probability P(94 Y, 95 G | Y, G) = P(94 Y, 95 G) / (P(94 Y, 95 G)+P(95 Y, 94 G))

P(94 Y, 95 G) = 0.2*0.2 = 0.04

P(95 Y, 94 G) = 0.1*0.14 = 0.014

Hence, the probability that the yellow one came from the 1994 bag is given by P(94 Y, 95 G | Y, G) = 20/27 = 0.741

Jupyter notebooks

Jupyter notebooks are useful for tinkering because they're very readable, but research projects that require more abstraction (more lines of code, or cyclomatic complexity, or Halstead complexity, or etc.) should be coded in Python scripts using an object-oriented programming paradigm in an integrated development environment with a more full suite of features. For this course, the Lab$n$_student.ipynb notebooks provided are sufficient.

Via a JupyterHub hosted by UCLA's IDRE, your notebooks will be in your own Docker container, where all the libraries you will need for the assignments should be installed. We recommend Chrome for accessing JupyterHub.

Tips:

- Save your notebook frequently (CMD+S on Mac.)
- Be careful about the order in which you run cells.

Resources:

- Assignment notebooks will contain some useful links.
- Stackoverflow is a great resource for troubleshooting.
- This book contains many good introductions to topics we'll discuss in the class, and it includes Python code written in Jupyter Notebooks.
- Here is a Jupyter Notebook cheatsheet.

Two Useful Commands

The code in the cell below imports a package. You can run the cell by clicking the button at the top of this webpage, or using the shortcuts CTRL/CMD+ENTER or SHIFT+ENTER

to run the cell and keep the cell selected or to run the cell then select the next cell, respectively. Also, here is a link to the NumPy documentation.

In [2]:
```
from numpy.testing import assert_array_almost_equal, assert_almost_equal, as
import numpy as np
```

Using an exclamation point, one can run shell commands inside of a Jupyter Notebook cell. For example, the following cell issues the `pwd` command, which prints the working directory.

In [3]:
```
! pwd
```

```
/Users/ameygaikwad/Library/Mobile Documents/com~apple~CloudDocs/Docs/Quarter
s/Winter 2026
```

Question 3 (1 pt)

Run the `%magic` command, then try out one of the commands listed in the resulting display. Make sure to delete the output cell before submitting the lab due to the significant length.

Your solution:

In [4]:
```
%magic
```

```
IPython's 'magic' functions
===========================

The magic function system provides a series of functions which allow you to
control the behavior of IPython itself, plus a lot of system-type
features. There are two kinds of magics, line-oriented and cell-oriented.

Line magics are prefixed with the % character and work much like OS
command-line calls: they get as an argument the rest of the line, where
arguments are passed without parentheses or quotes.  For example, this will
time the given statement::

        %timeit range(1000)

Cell magics are prefixed with a double %%, and they are functions that get a
s
an argument not only the rest of the line, but also the lines below it in a
separate argument.  These magics are called with two arguments: the rest of
the
call line and the body of the cell, consisting of the lines below the first.
For example::

        %%timeit x = numpy.random.randn((100, 100))
        numpy.linalg.svd(x)

will time the execution of the numpy svd routine, running the assignment of
x
as part of the setup phase, which is not timed.

In a line-oriented client (the terminal or Qt console IPython), starting a n
ew
input with %% will automatically enter cell mode, and IPython will continue
reading input until a blank line is given.  In the notebook, simply type the
whole cell as one entity, but keep in mind that the %% escape can only be at
the very start of the cell.

NOTE: If you have 'automagic' enabled (via the command line option or with t
he
%automagic function), you don't need to type in the % explicitly for line
magics; cell magics always require an explicit '%%' escape.  By default,
IPython ships with automagic on, so you should only rarely need the % escap
e.

Example: typing '%cd mydir' (without the quotes) changes your working direct
ory
to 'mydir', if it exists.

For a list of the available magic functions, use %lsmagic. For a description
of any of them, type %magic_name?, e.g. '%cd?'.

Currently the magic system has the following functions:
%alias:
    Define an alias for a system command.

    '%alias alias_name cmd' defines 'alias_name' as an alias for 'cmd'
```

```
Then, typing 'alias_name params' will execute the system command 'cmd
params' (from your underlying operating system).

Aliases have lower precedence than magic functions and Python normal
variables, so if 'foo' is both a Python variable and an alias, the
alias can not be executed until 'del foo' removes the Python variable.

You can use the %l specifier in an alias definition to represent the
whole line when the alias is called.  For example::

  In [2]: alias bracket echo "Input in brackets: <%l>"
  In [3]: bracket hello world
  Input in brackets: <hello world>

You can also define aliases with parameters using %s specifiers (one
per parameter)::

  In [1]: alias parts echo first %s second %s
  In [2]: %parts A B
  first A second B
  In [3]: %parts A
  Incorrect number of arguments: 2 expected.
  parts is an alias to: 'echo first %s second %s'

Note that %l and %s are mutually exclusive.  You can only use one or
the other in your aliases.

Aliases expand Python variables just like system calls using ! or !!
do: all expressions prefixed with '$' get expanded.  For details of
the semantic rules, see PEP-215:
https://peps.python.org/pep-0215/.  This is the library used by
IPython for variable expansion.  If you want to access a true shell
variable, an extra $ is necessary to prevent its expansion by
IPython::

  In [6]: alias show echo
  In [7]: PATH='A Python string'
  In [8]: show $PATH
  A Python string
  In [9]: show $$PATH
  /usr/local/lf9560/bin:/usr/local/intel/compiler70/ia32/bin:...

You can use the alias facility to access all of $PATH.  See the %rehashx
function, which automatically creates aliases for the contents of your
$PATH.

If called with no parameters, %alias prints the current alias table
for your system.  For posix systems, the default aliases are 'cat',
'cp', 'mv', 'rm', 'rmdir', and 'mkdir', and other platform-specific
aliases are added.  For windows-based systems, the default aliases are
'copy', 'ddir', 'echo', 'ls', 'ldir', 'mkdir', 'ren', and 'rmdir'.

You can see the definition of alias by adding a question mark in the
end::

  In [1]: cat?
```

```
      Repr: <alias cat for 'cat'>
%alias_magic:
    ::

      %alias_magic [-l] [-c] [-p PARAMS] name target

    Create an alias for an existing line or cell magic.

    Examples
    --------
    ::

      In [1]: %alias_magic t timeit
      Created `%t` as an alias for `%timeit`.
      Created `%%t` as an alias for `%%timeit`.

      In [2]: %t -n1 pass
      107 ns ± 43.6 ns per loop (mean ± std. dev. of 7 runs, 1 loop each)

      In [3]: %%t -n1
         ...: pass
         ...:
      107 ns ± 58.3 ns per loop (mean ± std. dev. of 7 runs, 1 loop each)

      In [4]: %alias_magic --cell whereami pwd
      UsageError: Cell magic function `%%pwd` not found.
      In [5]: %alias_magic --line whereami pwd
      Created `%whereami` as an alias for `%pwd`.

      In [6]: %whereami
      Out[6]: '/home/testuser'

      In [7]: %alias_magic h history -p "-l 30" --line
      Created `%h` as an alias for `%history -l 30`.

    positional arguments:
      name                  Name of the magic to be created.
      target                Name of the existing line or cell magic.

    options:
      -l, --line            Create a line magic alias.
      -c, --cell            Create a cell magic alias.
      -p PARAMS, --params PARAMS
                            Parameters passed to the magic function.
%autoawait:

    Allow to change the status of the autoawait option.

    This allow you to set a specific asynchronous code runner.

    If no value is passed, print the currently used asynchronous integration
    and whether it is activated.

    It can take a number of value evaluated in the following order:

    - False/false/off deactivate autoawait integration
```

```
    - True/true/on activate autoawait integration using configured default
      loop
    - asyncio/curio/trio activate autoawait integration and use integration
      with said library.

    - `sync` turn on the pseudo-sync integration (mostly used for
      `IPython.embed()` which does not run IPython with a real eventloop and
      deactivate running asynchronous code. Turning on Asynchronous code wit
h
      the pseudo sync loop is undefined behavior and may lead IPython to cra
sh.

    If the passed parameter does not match any of the above and is a python
    identifier, get said object from user namespace and set it as the
    runner, and activate autoawait.

    If the object is a fully qualified object name, attempt to import it and
    set it as the runner, and activate autoawait.

    The exact behavior of autoawait is experimental and subject to change
    across version of IPython and Python.
%autocall:
    Make functions callable without having to type parentheses.

    Usage:

       %autocall [mode]

    The mode can be one of: 0->Off, 1->Smart, 2->Full.  If not given, the
    value is toggled on and off (remembering the previous state).

    In more detail, these values mean:

    0 -> fully disabled

    1 -> active, but do not apply if there are no arguments on the line.

    In this mode, you get::

      In [1]: callable
      Out[1]: <built-in function callable>

      In [2]: callable 'hello'
      ------> callable('hello')
      Out[2]: False

    2 -> Active always.  Even if no arguments are present, the callable
    object is called::

      In [2]: float
      ------> float()
      Out[2]: 0.0

    Note that even with autocall off, you can still use '/' at the start of
    a line to treat the first argument on the command line as a function
    and add parentheses to it::
```

```
      In [8]: /str 43
      ------> str(43)
      Out[8]: '43'

    # all-random (note for auto-testing)
%automagic:
    Make magic functions callable without having to type the initial %.

    Without arguments toggles on/off (when off, you must call it as
    %automagic, of course).  With arguments it sets the value, and you can
    use any of (case insensitive):

     - on, 1, True: to activate

     - off, 0, False: to deactivate.

    Note that magic functions have lowest priority, so if there's a
    variable whose name collides with that of a magic fn, automagic won't
    work for that function (you get the variable instead). However, if you
    delete the variable (del var), the previously shadowed magic function
    becomes visible to automagic again.
%autosave:
    Set the autosave interval in the notebook (in seconds).

    The default value is 120, or two minutes.
    ``%autosave 0`` will disable autosave.

    This magic only has an effect when called from the notebook interface.
    It has no effect when called in a startup file.
%bookmark:
    Manage IPython's bookmark system.

    %bookmark <name>       - set bookmark to current dir
    %bookmark <name> <dir> - set bookmark to <dir>
    %bookmark -l           - list all bookmarks
    %bookmark -d <name>    - remove bookmark
    %bookmark -r           - remove all bookmarks

    You can later on access a bookmarked folder with::

      %cd -b <name>

    or simply '%cd <name>' if there is no directory called <name> AND
    there is such a bookmark defined.

    Your bookmarks persist through IPython sessions, but they are
    associated with each profile.
%cat:
    Alias for `!cat`
%cd:
    Change the current working directory.

    This command automatically maintains an internal list of directories
    you visit during your IPython session, in the variable ``_dh``. The
    command :magic:`%dhist` shows this history nicely formatted. You can
```

```
    also do ``cd -<tab>`` to see directory history conveniently.
    Usage:

      - ``cd 'dir'``: changes to directory 'dir'.
      - ``cd -``: changes to the last visited directory.
      - ``cd -<n>``: changes to the n-th directory in the directory history.
      - ``cd --foo``: change to directory that matches 'foo' in history
      - ``cd -b <bookmark_name>``: jump to a bookmark set by %bookmark
      - Hitting a tab key after ``cd -b`` allows you to tab-complete
        bookmark names.

      .. note::
        ``cd <bookmark_name>`` is enough if there is no directory
        ``<bookmark_name>``, but a bookmark with the name exists.

    Options:

    -q              Be quiet. Do not print the working directory after the
                    cd command is executed. By default IPython's cd
                    command does print this directory, since the default
                    prompts do not display path information.

    .. note::
       Note that ``!cd`` doesn't work for this purpose because the shell
       where ``!command`` runs is immediately discarded after executing
       'command'.

    Examples
    --------
    ::

      In [10]: cd parent/child
      /home/tsuser/parent/child
%clear:
    Clear the terminal.
%code_wrap:
    ::

      %code_wrap [--remove] [--list] [--list-all] [name]

    Simple magic to quickly define a code transformer for all IPython's futu
re input.

    ``__code__`` and ``__ret__`` are special variable that represent the cod
e to run
    and the value of the last expression of ``__code__`` respectively.

    Examples
    --------

    .. ipython::

        In [1]: %%code_wrap before_after
           ...: print('before')
           ...: __code__
           ...: print('after')
```

```
            ...: __ret__

        In [2]: 1
        before
        after
        Out[2]: 1

        In [3]: %code_wrap --list
        before_after

        In [4]: %code_wrap --list-all
        before_after :
            print('before')
            __code__
            print('after')
            __ret__

        In [5]: %code_wrap --remove before_after

    positional arguments:
      name

    options:
      --remove    remove the current transformer
      --list      list existing transformers name
      --list-all  list existing transformers name and code template
%colors:
    Switch color scheme for prompts, info system and exception handlers.

    Currently implemented schemes: NoColor, Linux, LightBG.

    Color scheme names are not case-sensitive.

    Examples
    --------
    To get a plain black and white terminal::

      %colors nocolor
%conda:
    Run the conda package manager within the current kernel.

    Usage:
      %conda install [pkgs]
%config:
    configure IPython

        %config Class[.trait=value]

    This magic exposes most of the IPython config system. Any
    Configurable class should be able to be configured with the simple
    line::

        %config Class.trait=value

    Where `value` will be resolved in the user's namespace, if it is an
    expression or variable name.
```

```
    Examples
    --------

    To see what classes are available for config, pass no arguments::

        In [1]: %config
        Available objects for config:
            AliasManager
            DisplayFormatter
            HistoryManager
            IPCompleter
            LoggingMagics
            MagicsManager
            OSMagics
            PrefilterManager
            ScriptMagics
            TerminalInteractiveShell

    To view what is configurable on a given class, just pass the class
    name::

        In [2]: %config LoggingMagics
        LoggingMagics(Magics) options
        ---------------------------
        LoggingMagics.quiet=<Bool>
            Suppress output of log state when logging is enabled
            Current: False

    but the real use is in setting values::

        In [3]: %config LoggingMagics.quiet = True

    and these values are read from the user_ns if they are variables::

        In [4]: feeling_quiet=False

        In [5]: %config LoggingMagics.quiet = feeling_quiet
%connect_info:
    Print information for connecting other clients to this kernel

    It will print the contents of this session's connection file, as well as
    shortcuts for local clients.

    In the simplest case, when called from the most recently launched kerne
l,
    secondary clients can be connected, simply with:

    $> jupyter <app> --existing
%cp:
    Alias for `!cp`
%debug:
    ::

      %debug [--breakpoint FILE:LINE] [statement ...]
```

```
    Activate the interactive debugger.

    This magic command support two ways of activating debugger.
    One is to activate debugger before executing code.  This way, you
    can set a break point, to step through the code from the point.
    You can use this mode by giving statements to execute and optionally
    a breakpoint.

    The other one is to activate debugger in post-mortem mode.  You can
    activate this mode simply running %debug without any argument.
    If an exception has just occurred, this lets you inspect its stack
    frames interactively.  Note that this will always work only on the last
    traceback that occurred, so you must call this quickly after an
    exception that you wish to inspect has fired, because if another one
    occurs, it clobbers the previous one.

    If you want IPython to automatically do this on every exception, see
    the %pdb magic for more details.

    .. versionchanged:: 7.3
        When running code, user variables are no longer expanded,
        the magic line is always left unmodified.

    positional arguments:
      statement             Code to run in debugger. You can omit this in ce
ll
                            magic mode.

    options:
      --breakpoint <FILE:LINE>, -b <FILE:LINE>
                            Set break point at LINE in FILE.
%dhist:
    Print your history of visited directories.

    %dhist       -> print full history
    %dhist n     -> print last n entries only
    %dhist n1 n2 -> print entries between n1 and n2 (n2 not included)

    This history is automatically maintained by the %cd command, and
    always available as the global list variable _dh. You can use %cd -<n>
    to go to directory number <n>.

    Note that most of time, you should view directory history by entering
    cd -<TAB>.
%dirs:
    Return the current directory stack.
%doctest_mode:
    Toggle doctest mode on and off.

    This mode is intended to make IPython behave as much as possible like a
    plain Python shell, from the perspective of how its prompts, exceptions
    and output look.  This makes it easy to copy and paste parts of a
    session into doctests.  It does so by:

    - Changing the prompts to the classic ``>>>`` ones.
    - Changing the exception reporting mode to 'Plain'.
```

```
    - Disabling pretty-printing of output.

    Note that IPython also supports the pasting of code snippets that have
    leading '>>>' and '...' prompts in them.  This means that you can paste
    doctests from files or docstrings (even if they have leading
    whitespace), and the code will execute correctly.  You can then use
    '%history -t' to see the translated history; this will give you the
    input after removal of all the leading prompts and whitespace, which
    can be pasted back into an editor.

    With these features, you can switch into this mode easily whenever you
    need to do testing and changes to doctests, without having to leave
    your existing IPython session.
%ed:
    Alias for `%edit`.
%edit:
    Bring up an editor and execute the resulting code.

    Usage:
      %edit [options] [args]

    %edit runs an external text editor. You will need to set the command for
    this editor via the ``TerminalInteractiveShell.editor`` option in your
    configuration file before it will work.

    This command allows you to conveniently edit multi-line code right in
    your IPython session.

    If called without arguments, %edit opens up an empty editor with a
    temporary file and will execute the contents of this file when you
    close it (don't forget to save it!).

    Options:

    -n <number>
      Open the editor at a specified line number. By default, the IPython
      editor hook uses the unix syntax 'editor +N filename', but you can
      configure this by providing your own modified hook if your favorite
      editor supports line-number specifications with a different syntax.

    -p
      Call the editor with the same data as the previous time it was used,
      regardless of how long ago (in your current session) it was.

    -r
      Use 'raw' input. This option only applies to input taken from the
      user's history.  By default, the 'processed' history is used, so that
      magics are loaded in their transformed version to valid Python.  If
      this option is given, the raw input as typed as the command line is
      used instead.  When you exit the editor, it will be executed by
      IPython's own processor.

    Arguments:

    If arguments are given, the following possibilities exist:
```

```
    - The arguments are numbers or pairs of colon-separated numbers (like
      1 4:8 9). These are interpreted as lines of previous input to be
      loaded into the editor. The syntax is the same of the %macro command.

    - If the argument doesn't start with a number, it is evaluated as a
      variable and its contents loaded into the editor. You can thus edit
      any string which contains python code (including the result of
      previous edits).

    - If the argument is the name of an object (other than a string),
      IPython will try to locate the file where it was defined and open the
      editor at the point where it is defined. You can use ``%edit function`
`
      to load an editor exactly at the point where 'function' is defined,
      edit it and have the file be executed automatically.

      If the object is a macro (see %macro for details), this opens up your
      specified editor with a temporary file containing the macro's data.
      Upon exit, the macro is reloaded with the contents of the file.

      Note: opening at an exact line is only supported under Unix, and some
      editors (like kedit and gedit up to Gnome 2.8) do not understand the
      '+NUMBER' parameter necessary for this feature. Good editors like
      (X)Emacs, vi, jed, pico and joe all do.

    - If the argument is not found as a variable, IPython will look for a
      file with that name (adding .py if necessary) and load it into the
      editor. It will execute its contents with execfile() when you exit,
      loading any code in the file into your interactive namespace.

    Unlike in the terminal, this is designed to use a GUI editor, and we do
    not know when it has closed. So the file you edit will not be
    automatically executed or printed.

    Note that %edit is also available through the alias %ed.
%env:
    Get, set, or list environment variables.

    Usage:

      :``%env``: lists all environment variables/values
      :``%env var``: get value for var
      :``%env var val``: set value for var
      :``%env var=val``: set value for var
      :``%env var=$val``: set value for var, using python expansion if possi
ble
%gui:
    Enable or disable IPython GUI event loop integration.

    %gui [GUINAME]

    This magic replaces IPython's threaded shells that were activated
    using the (pylab/wthread/etc.) command line flags.  GUI toolkits
    can now be enabled at runtime and keyboard
    interrupts should work without any problems.  The following toolkits
    are supported:  wxPython, PyQt4, PyGTK, Tk and Cocoa (OSX)::
```

```
            %gui wx      # enable wxPython event loop integration
            %gui qt      # enable PyQt/PySide event loop integration
                         # with the latest version available.
            %gui qt6     # enable PyQt6/PySide6 event loop integration
            %gui qt5     # enable PyQt5/PySide2 event loop integration
            %gui gtk     # enable PyGTK event loop integration
            %gui gtk3    # enable Gtk3 event loop integration
            %gui gtk4    # enable Gtk4 event loop integration
            %gui tk      # enable Tk event loop integration
            %gui osx     # enable Cocoa event loop integration
                         # (requires %matplotlib 1.1)
            %gui         # disable all event loop integration

        WARNING:  after any of these has been called you can simply create
        an application object, but DO NOT start the event loop yourself, as
        we have already handled that.
    %hist:
        Alias for `%history`.
    %history:
        ::

          %history [-n] [-o] [-p] [-t] [-f FILENAME] [-g [PATTERN ...]]
                       [-l [LIMIT]] [-u]
                       [range ...]

        Print input history (_i<n> variables), with most recent last.

        By default, input history is printed without line numbers so it can be
        directly pasted into an editor. Use -n to show them.

        By default, all input history from the current session is displayed.
        Ranges of history can be indicated using the syntax:

        ``4``
            Line 4, current session
        ``4-6``
            Lines 4-6, current session
        ``243/1-5``
            Lines 1-5, session 243
        ``~2/7``
            Line 7, session 2 before current
        ``~8/1-~6/5``
            From the first line of 8 sessions ago, to the fifth line of 6
            sessions ago.

        Multiple ranges can be entered, separated by spaces

        The same syntax is used by %macro, %save, %edit, %rerun

        Examples
        --------
        ::

          In [6]: %history -n 4-6
          4:a = 12
```

```
      5:print(a**2)
      6:%history -n 4-6

    positional arguments:
      range

    options:
      -n                    print line numbers for each input. This feature is
only
                            available if numbered prompts are in use.
      -o                    also print outputs for each input.
      -p                    print classic '>>>' python prompts before each inp
ut.
                            This is useful for making documentation, and in
                            conjunction with -o, for producing doctest-ready o
utput.
      -t                    print the 'translated' history, as IPython underst
ands
                            it. IPython filters your input and converts it all
into
                            valid Python source before executing it (things li
ke
                            magics or aliases are turned into function calls,
for
                            example). With this option, you'll see the native
                            history instead of the user-entered version: '%cd
/'
                            will be seen as 'get_ipython().run_line_magic("c
d",
                            "/")' instead of '%cd /'.
      -f FILENAME           FILENAME: instead of printing the output to the sc
reen,
                            redirect it to the given file. The file is always
                            overwritten, though *when it can*, IPython asks fo
r
                            confirmation first. In particular, running the com
mand
                            'history -f FILENAME' from the IPython Notebook
                            interface will replace FILENAME even if it already
                            exists *without* confirmation.
      -g <[PATTERN ...]>    treat the arg as a glob pattern to search for in
(full)
                            history. This includes the saved history (almost a
ll
                            commands ever written). The pattern may contain
'?' to
                            match one unknown character and '*' to match any n
umber
                            of unknown characters. Use '%hist -g' to show full
saved
                            history (may be very long).
      -l <[LIMIT]>          get the last n lines from all sessions. Specify n
as a
                            single arg, or the default is the last 10 lines.
      -u                    when searching history using `-g`, show only uniqu
e
```

```
                            history.
%killbgscripts:
    Kill all BG processes started by %%script and its family.
%ldir:
    Alias for `!ls -F -G -l %l | grep /$`
%less:
    Show a file through the pager.

    Files ending in .py are syntax-highlighted.
%lf:
    Alias for `!ls -F -l -G %l | grep ^-`
%lk:
    Alias for `!ls -F -l -G %l | grep ^l`
%ll:
    Alias for `!ls -F -l -G`
%load:
    Load code into the current frontend.

    Usage:
      %load [options] source

      where source can be a filename, URL, input history range, macro, or
      element in the user namespace

    If no arguments are given, loads the history of this session up to this
    point.

    Options:

      -r <lines>: Specify lines or ranges of lines to load from the source.
      Ranges could be specified as x-y (x..y) or in python-style x:y
      (x..(y-1)). Both limits x and y can be left blank (meaning the
      beginning and end of the file, respectively).

      -s <symbols>: Specify function or classes to load from python source.

      -y : Don't ask confirmation for loading source above 200 000 character
s.

      -n : Include the user's namespace when searching for source code.

    This magic command can either take a local filename, a URL, an history
    range (see %history) or a macro as argument, it will prompt for
    confirmation before loading source with more than 200 000 characters, un
less
    -y flag is passed or if the frontend does not support raw_input::

    %load
    %load myscript.py
    %load 7-27
    %load myMacro
    %load http://www.example.com/myscript.py
    %load -r 5-10 myscript.py
    %load -r 10-20,30,40: foo.py
    %load -s MyClass,wonder_function myscript.py
    %load -n MyClass
```

```
    %load -n my_module.wonder_function
%load_ext:
    Load an IPython extension by its module name.
%loadpy:
    Alias of `%load`

    `%loadpy` has gained some flexibility and dropped the requirement of a
`.py`
    extension. So it has been renamed simply into %load. You can look at
    `%load`'s docstring for more info.
%logoff:
    Temporarily stop logging.

    You must have previously started logging.
%logon:
    Restart logging.

    This function is for restarting logging which you've temporarily
    stopped with %logoff. For starting logging for the first time, you
    must use the %logstart function, which allows you to specify an
    optional log filename.
%logstart:
    Start logging anywhere in a session.

    %logstart [-o|-r|-t|-q] [log_name [log_mode]]

    If no name is given, it defaults to a file named 'ipython_log.py' in you
r
    current directory, in 'rotate' mode (see below).

    '%logstart name' saves to file 'name' in 'backup' mode.  It saves your
    history up to that point and then continues logging.

    %logstart takes a second optional parameter: logging mode. This can be o
ne
    of (note that the modes are given unquoted):

    append
        Keep logging at the end of any existing file.

    backup
        Rename any existing file to name~ and start name.

    global
        Append to  a single logfile in your home directory.

    over
        Overwrite any existing log.

    rotate
        Create rotating logs: name.1~, name.2~, etc.

    Options:

      -o
        log also IPython's output. In this mode, all commands which
```

```
          generate an Out[NN] prompt are recorded to the logfile, right after
          their corresponding input line. The output lines are always
          prepended with a '#[Out]# ' marker, so that the log remains valid
          Python code.

        Since this marker is always the same, filtering only the output from
        a log is very easy, using for example a simple awk call::

          awk -F'#\[Out\]# ' '{if($2) {print $2}}' ipython_log.py

        -r
          log 'raw' input.  Normally, IPython's logs contain the processed
          input, so that user lines are logged in their final form, converted
          into valid Python.  For example, %Exit is logged as
          _ip.run_line_magic("Exit").  If the -r flag is given, all input is l
ogged
          exactly as typed, with no transformations applied.

        -t
          put timestamps before each input line logged (these are put in
          comments).

        -q
          suppress output of logstate message when logging is invoked
%logstate:
    Print the status of the logging system.
%logstop:
    Fully stop logging and close log file.

    In order to start logging again, a new %logstart call needs to be made,
    possibly (though not necessarily) with a new filename, mode and other
    options.
%ls:
    Alias for `!ls -F -G`
%lsmagic:
    List currently available magic functions.
%lx:
    Alias for `!ls -F -l -G %l | grep ^-..x`
%macro:
    Define a macro for future re-execution. It accepts ranges of history,
    filenames or string objects.

    Usage:
      %macro [options] name n1-n2 n3-n4 ... n5 .. n6 ...

    Options:

      -r: use 'raw' input.  By default, the 'processed' history is used,
      so that magics are loaded in their transformed version to valid
      Python.  If this option is given, the raw input as typed at the
      command line is used instead.

      -q: quiet macro definition.  By default, a tag line is printed
      to indicate the macro has been created, and then the contents of
      the macro are printed.  If this option is given, then no printout
      is produced once the macro is created.
```

```
    This will define a global variable called `name` which is a string
    made of joining the slices and lines you specify (n1,n2,... numbers
    above) from your input history into a single string. This variable
    acts like an automatic function which re-executes those lines as if
    you had typed them. You just type 'name' at the prompt and the code
    executes.

    The syntax for indicating input ranges is described in %history.

    Note: as a 'hidden' feature, you can also use traditional python slice
    notation, where N:M means numbers N through M-1.

    For example, if your history contains (print using %hist -n )::

      44: x=1
      45: y=3
      46: z=x+y
      47: print(x)
      48: a=5
      49: print('x',x,'y',y)

    you can create a macro with lines 44 through 47 (included) and line 49
    called my_macro with::

      In [55]: %macro my_macro 44-47 49

    Now, typing `my_macro` (without quotes) will re-execute all this code
    in one pass.

    You don't need to give the line-numbers in order, and any given line
    number can appear multiple times. You can assemble macros with any
    lines from your input history in any order.

    The macro is a simple object which holds its value in an attribute,
    but IPython's display system checks for macros and executes them as
    code instead of printing them when you type their name.

    You can view a macro's contents by explicitly printing it with::

      print(macro_name)
%magic:
    Print information about the magic function system.

    Supported formats: -latex, -brief, -rest
%mamba:
    Run the mamba package manager within the current kernel.

    Usage:
      %mamba install [pkgs]
%man:
    Find the man page for the given command and display in pager.
%matplotlib:
    ::

      %matplotlib [-l] [gui]
```

```
    Set up matplotlib to work interactively.

    This function lets you activate matplotlib interactive support
    at any point during an IPython session. It does not import anything
    into the interactive namespace.

    If you are using the inline matplotlib backend in the IPython Notebook
    you can set which figure formats are enabled using the following::

        In [1]: from matplotlib_inline.backend_inline import set_matplotlib_
formats

        In [2]: set_matplotlib_formats('pdf', 'svg')

    The default for inline figures sets `bbox_inches` to 'tight'. This can
    cause discrepancies between the displayed image and the identical
    image created using `savefig`. This behavior can be disabled using the
    `%config` magic::

        In [3]: %config InlineBackend.print_figure_kwargs = {'bbox_inches':N
one}

    In addition, see the docstrings of
    `matplotlib_inline.backend_inline.set_matplotlib_formats` and
    `matplotlib_inline.backend_inline.set_matplotlib_close` for more informa
tion on
    changing additional behaviors of the inline backend.

    Examples
    --------
    To enable the inline backend for usage with the IPython Notebook::

        In [1]: %matplotlib inline

    In this case, where the matplotlib default is TkAgg::

        In [2]: %matplotlib
        Using matplotlib backend: TkAgg

    But you can explicitly request a different GUI backend::

        In [3]: %matplotlib qt

    You can list the available backends using the -l/--list option::

       In [4]: %matplotlib --list
       Available matplotlib backends: ['osx', 'qt4', 'qt5', 'gtk3', 'gtk4',
'notebook', 'wx', 'qt', 'nbagg',
       'gtk', 'tk', 'inline']

    positional arguments:
      gui         Name of the matplotlib backend to use such as 'qt' or 'wid
get'.
                  If given, the corresponding matplotlib backend is used,
                  otherwise it will be matplotlib's default (which you can s
```

```
et in
                  your matplotlib config file).

    options:
      -l, --list  Show available matplotlib backends
%micromamba:
    Run the conda package manager within the current kernel.

    Usage:
      %micromamba install [pkgs]
%mkdir:
    Alias for `!mkdir`
%more:
    Show a file through the pager.

    Files ending in .py are syntax-highlighted.
%mv:
    Alias for `!mv`
%notebook:
    ::

      %notebook filename

    Export and convert IPython notebooks.

    This function can export the current IPython history to a notebook file.
    For example, to export the history to "foo.ipynb" do "%notebook foo.ipyn
b".

    positional arguments:
      filename  Notebook name or filename
%page:
    Pretty print the object and display it through a pager.

    %page [options] OBJECT

    If no object is given, use _ (last output).

    Options:

      -r: page str(object), don't pretty-print it.
%pastebin:
    Upload code to dpaste.com, returning the URL.

    Usage:
      %pastebin [-d "Custom description"][-e 24] 1-7

    The argument can be an input history range, a filename, or the name of a
    string or macro.

    If no arguments are given, uploads the history of this session up to
    this point.

    Options:

      -d: Pass a custom description. The default will say
```

```
          "Pasted from IPython".
        -e: Pass number of days for the link to be expired.
          The default will be 7 days.
%pdb:
    Control the automatic calling of the pdb interactive debugger.

    Call as '%pdb on', '%pdb 1', '%pdb off' or '%pdb 0'. If called without
    argument it works as a toggle.

    When an exception is triggered, IPython can optionally call the
    interactive pdb debugger after the traceback printout. %pdb toggles
    this feature on and off.

    The initial state of this feature is set in your configuration
    file (the option is ``InteractiveShell.pdb``).

    If you want to just activate the debugger AFTER an exception has fired,
    without having to type '%pdb on' and rerunning your code, you can use
    the %debug magic.
%pdef:
    Print the call signature for any callable object.

    If the object is a class, print the constructor information.

    Examples
    --------
    ::

      In [3]: %pdef urllib.urlopen
      urllib.urlopen(url, data=None, proxies=None)
%pdoc:
    Print the docstring for an object.

    If the given object is a class, it will print both the class and the
    constructor docstrings.
%pfile:
    Print (or run through pager) the file where an object is defined.

    The file opens at the line where the object definition begins. IPython
    will honor the environment variable PAGER if set, and otherwise will
    do its best to print the file in a convenient form.

    If the given argument is not an object currently defined, IPython will
    try to interpret it as a filename (automatically adding a .py extension
    if needed). You can thus use %pfile as a syntax highlighting code
    viewer.
%pinfo:
    Provide detailed information about an object.

    '%pinfo object' is just a synonym for object? or ?object.
%pinfo2:
    Provide extra detailed information about an object.

    '%pinfo2 object' is just a synonym for object?? or ??object.
%pip:
    Run the pip package manager within the current kernel.
```

```
    Usage:
      %pip install [pkgs]
%popd:
    Change to directory popped off the top of the stack.
%pprint:
    Toggle pretty printing on/off.
%precision:
    Set floating point precision for pretty printing.

    Can set either integer precision or a format string.

    If numpy has been imported and precision is an int,
    numpy display precision will also be set, via ``numpy.set_printoptions`
`.

    If no argument is given, defaults will be restored.

    Examples
    --------
    ::

        In [1]: from math import pi

        In [2]: %precision 3
        Out[2]: '%.3f'

        In [3]: pi
        Out[3]: 3.142

        In [4]: %precision %i
        Out[4]: '%i'

        In [5]: pi
        Out[5]: 3

        In [6]: %precision %e
        Out[6]: '%e'

        In [7]: pi**10
        Out[7]: 9.364805e+04

        In [8]: %precision
        Out[8]: '%r'

        In [9]: pi**10
        Out[9]: 93648.047476082982
%prun:
    Run a statement through the python code profiler.

    **Usage, in line mode:**

      %prun [options] statement

    **Usage, in cell mode:**
```

```
  %%prun [options] [statement]

  code...

  code...

In cell mode, the additional code lines are appended to the (possibly
empty) statement in the first line.  Cell mode allows you to easily
profile multiline blocks without having to put them in a separate
function.

The given statement (which doesn't require quote marks) is run via the
python profiler in a manner similar to the profile.run() function.
Namespaces are internally managed to work correctly; profile.run
cannot be used in IPython because it makes certain assumptions about
namespaces which do not hold under IPython.

Options:

-l <limit>
  you can place restrictions on what or how much of the
  profile gets printed. The limit value can be:

     * A string: only information for function names containing this str
ing
       is printed.

     * An integer: only these many lines are printed.

     * A float (between 0 and 1): this fraction of the report is printed
       (for example, use a limit of 0.4 to see the topmost 40% only).

  You can combine several limits with repeated use of the option. For
  example, ``-l __init__ -l 5`` will print only the topmost 5 lines of
  information about class constructors.

-r
  return the pstats.Stats object generated by the profiling. This
  object has all the information about the profile in it, and you can
  later use it for further analysis or in other functions.

-s <key>
  sort profile by given key. You can provide more than one key
  by using the option several times: '-s key1 -s key2 -s key3...'. The
  default sorting key is 'time'.

  The following is copied verbatim from the profile documentation
  referenced below:

  When more than one key is provided, additional keys are used as
  secondary criteria when the there is equality in all keys selected
  before them.

  Abbreviations can be used for any key names, as long as the
  abbreviation is unambiguous.  The following are the keys currently
  defined:
```

```
      ============  ====================
      Valid Arg     Meaning
      ============  ====================
      "calls"       call count
      "cumulative"  cumulative time
      "file"        file name
      "module"      file name
      "pcalls"      primitive call count
      "line"        line number
      "name"        function name
      "nfl"         name/file/line
      "stdname"     standard name
      "time"        internal time
      ============  ====================

      Note that all sorts on statistics are in descending order (placing
      most time consuming items first), where as name, file, and line number
      searches are in ascending order (i.e., alphabetical). The subtle
      distinction between "nfl" and "stdname" is that the standard name is a
      sort of the name as printed, which means that the embedded line
      numbers get compared in an odd way.  For example, lines 3, 20, and 40
      would (if the file names were the same) appear in the string order
      "20" "3" and "40".  In contrast, "nfl" does a numeric compare of the
      line numbers.  In fact, sort_stats("nfl") is the same as
      sort_stats("name", "file", "line").

    -T <filename>
      save profile results as shown on screen to a text
      file. The profile is still shown on screen.

    -D <filename>
      save (via dump_stats) profile statistics to given
      filename. This data is in a format understood by the pstats module, an
d
      is generated by a call to the dump_stats() method of profile
      objects. The profile is still shown on screen.

    -q
      suppress output to the pager.  Best used with -T and/or -D above.

    If you want to run complete programs under the profiler's control, use
    ``%run -p [prof_opts] filename.py [args to program]`` where prof_opts
    contains profiler specific options as described here.

    You can read the complete documentation for the profile module with::

      In [1]: import profile; profile.help()

    .. versionchanged:: 7.3
        User variables are no longer expanded,
        the magic line is always left unmodified.
%psearch:
    Search for object in namespaces by wildcard.

    %psearch [options] PATTERN [OBJECT TYPE]
```

```
Note: ? can be used as a synonym for %psearch, at the beginning or at
the end: both a*? and ?a* are equivalent to '%psearch a*'.  Still, the
rest of the command line must be unchanged (options come first), so
for example the following forms are equivalent

%psearch -i a* function
-i a* function?
?-i a* function

Arguments:

  PATTERN

  where PATTERN is a string containing * as a wildcard similar to its
  use in a shell.  The pattern is matched in all namespaces on the
  search path. By default objects starting with a single _ are not
  matched, many IPython generated objects have a single
  underscore. The default is case insensitive matching. Matching is
  also done on the attributes of objects and not only on the objects
  in a module.

  [OBJECT TYPE]

  Is the name of a python type from the types module. The name is
  given in lowercase without the ending type, ex. StringType is
  written string. By adding a type here only objects matching the
  given type are matched. Using all here makes the pattern match all
  types (this is the default).

Options:

  -a: makes the pattern match even objects whose names start with a
  single underscore.  These names are normally omitted from the
  search.

  -i/-c: make the pattern case insensitive/sensitive.  If neither of
  these options are given, the default is read from your configuration
  file, with the option ``InteractiveShell.wildcards_case_sensitive``.
  If this option is not specified in your configuration file, IPython's
  internal default is to do a case sensitive search.

  -e/-s NAMESPACE: exclude/search a given namespace.  The pattern you
  specify can be searched in any of the following namespaces:
  'builtin', 'user', 'user_global','internal', 'alias', where
  'builtin' and 'user' are the search defaults.  Note that you should
  not use quotes when specifying namespaces.

  -l: List all available object types for object matching. This function
  can be used without arguments.

  'Builtin' contains the python module builtin, 'user' contains all
  user data, 'alias' only contain the shell aliases and no python
  objects, 'internal' contains objects used by IPython.  The
  'user_global' namespace is only used by embedded IPython instances,
  and it contains module-level globals.  You can add namespaces to the
```

```
      search with -s or exclude them with -e (these options can be given
      more than once).

    Examples
    --------
    ::

      %psearch a*            -> objects beginning with an a
      %psearch -e builtin a* -> objects NOT in the builtin space starting in
a
      %psearch a* function   -> all functions beginning with an a
      %psearch re.e*         -> objects beginning with an e in module re
      %psearch r*.e*         -> objects that start with e in modules startin
g in r
      %psearch r*.* string   -> all strings in modules beginning with r

    Case sensitive search::

      %psearch -c a*         list all object beginning with lower case a

    Show objects beginning with a single _::

      %psearch -a _*         list objects beginning with a single underscore

    List available objects::

      %psearch -l            list all available object types
%psource:
    Print (or run through pager) the source code for an object.
%pushd:
    Place the current dir on stack and change directory.

    Usage:
      %pushd ['dirname']
%pwd:
    Return the current working directory path.

    Examples
    --------
    ::

      In [9]: pwd
      Out[9]: '/home/tsuser/sprint/ipython'
%pycat:
    Show a syntax-highlighted file through a pager.

    This magic is similar to the cat utility, but it will assume the file
    to be Python source and will show it with syntax highlighting.

    This magic command can either take a local filename, an url,
    an history range (see %history) or a macro as argument.

    If no parameter is given, prints out history of current session up to
    this point. ::

    %pycat myscript.py
```

```
    %pycat 7-27
    %pycat myMacro
    %pycat http://www.example.com/myscript.py
%pylab:
    ::

      %pylab [--no-import-all] [gui]

    Load numpy and matplotlib to work interactively.

    This function lets you activate pylab (matplotlib, numpy and
    interactive support) at any point during an IPython session.

    %pylab makes the following imports::

        import numpy
        import matplotlib
        from matplotlib import pylab, mlab, pyplot
        np = numpy
        plt = pyplot

        from IPython.display import display
        from IPython.core.pylabtools import figsize, getfigs

        from pylab import *
        from numpy import *

    If you pass `--no-import-all`, the last two `*` imports will be exclude
d.

    See the %matplotlib magic for more details about activating matplotlib
    without affecting the interactive namespace.

    positional arguments:
      gui              Name of the matplotlib backend to use such as 'qt' or
                       'widget'. If given, the corresponding matplotlib back
end is
                       used, otherwise it will be matplotlib's default (whic
h you
                       can set in your matplotlib config file).

    options:
      --no-import-all  Prevent IPython from performing ``import *`` into the
                       interactive namespace. You can govern the default beh
avior
                       of this flag with the InteractiveShellApp.pylab_impor
t_all
                       configurable.
%qtconsole:
    Open a qtconsole connected to this kernel.

    Useful for connecting a qtconsole to running notebooks, for better
    debugging.
%quickref:
    Show a quick reference sheet
%recall:
```

```
    Repeat a command, or get command to input line for editing.

    %recall and %rep are equivalent.

    - %recall (no arguments):

    Place a string version of last computation result (stored in the
    special '_' variable) to the next input prompt. Allows you to create
    elaborate command lines without using copy-paste::

         In[1]: l = ["hei", "vaan"]
         In[2]: "".join(l)
        Out[2]: heivaan
         In[3]: %recall
         In[4]: heivaan_ <== cursor blinking

    %recall 45

    Place history line 45 on the next input prompt. Use %hist to find
    out the number.

    %recall 1-4

    Combine the specified lines into one cell, and place it on the next
    input prompt. See %history for the slice syntax.

    %recall foo+bar

    If foo+bar can be evaluated in the user namespace, the result is
    placed at the next input prompt. Otherwise, the history is searched
    for lines which contain that substring, and the most recent one is
    placed at the next input prompt.
%rehashx:
    Update the alias table with all executable files in $PATH.

    rehashx explicitly checks that every entry in $PATH is a file
    with execute access (os.X_OK).

    Under Windows, it checks executability as a match against a
    '|'-separated string of extensions, stored in the IPython config
    variable win_exec_ext.  This defaults to 'exe|com|bat'.

    This function also resets the root module cache of module completer,
    used on slow filesystems.
%reload_ext:
    Reload an IPython extension by its module name.
%rep:
    Alias for `%recall`.
%rerun:
    Re-run previous input

    By default, you can specify ranges of input history to be repeated
    (as with %history). With no arguments, it will repeat the last line.

    Options:
```

```
      -l <n> : Repeat the last n lines of input, not including the
      current command.

      -g foo : Repeat the most recent line which contains foo
%reset:
    Resets the namespace by removing all names defined by the user, if
    called without arguments, or by removing some types of objects, such
    as everything currently in IPython's In[] and Out[] containers (see
    the parameters for details).

    Parameters
    ----------
    -f
        force reset without asking for confirmation.
    -s
        'Soft' reset: Only clears your namespace, leaving history intact.
        References to objects may be kept. By default (without this option),
        we do a 'hard' reset, giving you a new session and removing all
        references to objects from the current session.
    --aggressive
        Try to aggressively remove modules from sys.modules ; this
        may allow you to reimport Python modules that have been updated and
        pick up changes, but can have unintended consequences.

    in
        reset input history
    out
        reset output history
    dhist
        reset directory history
    array
        reset only variables that are NumPy arrays

    See Also
    --------
    reset_selective : invoked as ``%reset_selective``

    Examples
    --------
    ::

      In [6]: a = 1

      In [7]: a
      Out[7]: 1

      In [8]: 'a' in get_ipython().user_ns
      Out[8]: True

      In [9]: %reset -f

      In [1]: 'a' in get_ipython().user_ns
      Out[1]: False

      In [2]: %reset -f in
      Flushing input history
```

```
      In [3]: %reset -f dhist in
      Flushing directory history
      Flushing input history

    Notes
    -----
    Calling this magic from clients that do not implement standard input,
    such as the ipython notebook interface, will reset the namespace
    without confirmation.
%reset_selective:
    Resets the namespace by removing names defined by the user.

    Input/Output history are left around in case you need them.

    %reset_selective [-f] regex

    No action is taken if regex is not included

    Options
      -f : force reset without asking for confirmation.

    See Also
    --------
    reset : invoked as ``%reset``

    Examples
    --------
    We first fully reset the namespace so your output looks identical to
    this example for pedagogical reasons; in practice you do not need a
    full reset::

      In [1]: %reset -f

    Now, with a clean namespace we can make a few variables and use
    ``%reset_selective`` to only delete names that match our regexp::

      In [2]: a=1; b=2; c=3; b1m=4; b2m=5; b3m=6; b4m=7; b2s=8

      In [3]: who_ls
      Out[3]: ['a', 'b', 'b1m', 'b2m', 'b2s', 'b3m', 'b4m', 'c']

      In [4]: %reset_selective -f b[2-3]m

      In [5]: who_ls
      Out[5]: ['a', 'b', 'b1m', 'b2s', 'b4m', 'c']

      In [6]: %reset_selective -f d

      In [7]: who_ls
      Out[7]: ['a', 'b', 'b1m', 'b2s', 'b4m', 'c']

      In [8]: %reset_selective -f c

      In [9]: who_ls
      Out[9]: ['a', 'b', 'b1m', 'b2s', 'b4m']
```

```
      In [10]: %reset_selective -f b

      In [11]: who_ls
      Out[11]: ['a']

    Notes
    -----
    Calling this magic from clients that do not implement standard input,
    such as the ipython notebook interface, will reset the namespace
    without confirmation.
%rm:
    Alias for `!rm`
%rmdir:
    Alias for `!rmdir`
%run:
    Run the named file inside IPython as a program.

    Usage::

      %run [-n -i -e -G]
           [( -t [-N<N>] | -d [-b<N>] | -p [profile options] )]
           ( -m mod | filename ) [args]

    The filename argument should be either a pure Python script (with
    extension ``.py``), or a file with custom IPython syntax (such as
    magics). If the latter, the file can be either a script with ``.ipy``
    extension, or a Jupyter notebook with ``.ipynb`` extension. When running
    a Jupyter notebook, the output from print statements and other
    displayed objects will appear in the terminal (even matplotlib figures
    will open, if a terminal-compliant backend is being used). Note that,
    at the system command line, the ``jupyter run`` command offers similar
    functionality for executing notebooks (albeit currently with some
    differences in supported options).

    Parameters after the filename are passed as command-line arguments to
    the program (put in sys.argv). Then, control returns to IPython's
    prompt.

    This is similar to running at a system prompt ``python file args``,
    but with the advantage of giving you IPython's tracebacks, and of
    loading all variables into your interactive namespace for further use
    (unless -p is used, see below).

    The file is executed in a namespace initially consisting only of
    ``__name__=='__main__'`` and sys.argv constructed as indicated. It thus
    sees its environment as if it were being run as a stand-alone program
    (except for sharing global objects such as previously imported
    modules). But after execution, the IPython interactive namespace gets
    updated with all variables defined in the program (except for __name__
    and sys.argv). This allows for very convenient loading of code for
    interactive work, while giving each program a 'clean sheet' to run in.

    Arguments are expanded using shell-like glob match.  Patterns
    '*', '?', '[seq]' and '[!seq]' can be used.  Additionally,
    tilde '~' will be expanded into user's home directory.  Unlike
```

```
real shells, quotation does not suppress expansions.  Use
*two* back slashes (e.g. ``\\*``) to suppress expansions.
To completely disable these expansions, you can use -G flag.

On Windows systems, the use of single quotes `'` when specifying
a file is not supported. Use double quotes `"`.

Options:

-n
  __name__ is NOT set to '__main__', but to the running file's name
  without extension (as python does under import).  This allows running
  scripts and reloading the definitions in them without calling code
  protected by an ``if __name__ == "__main__"`` clause.

-i
  run the file in IPython's namespace instead of an empty one. This
  is useful if you are experimenting with code written in a text editor
  which depends on variables defined interactively.

-e
  ignore sys.exit() calls or SystemExit exceptions in the script
  being run.  This is particularly useful if IPython is being used to
  run unittests, which always exit with a sys.exit() call.  In such
  cases you are interested in the output of the test results, not in
  seeing a traceback of the unittest module.

-t
  print timing information at the end of the run.  IPython will give
  you an estimated CPU time consumption for your script, which under
  Unix uses the resource module to avoid the wraparound problems of
  time.clock().  Under Unix, an estimate of time spent on system tasks
  is also given (for Windows platforms this is reported as 0.0).

If -t is given, an additional ``-N<N>`` option can be given, where <N>
must be an integer indicating how many times you want the script to
run.  The final timing report will include total and per run results.

For example (testing the script uniq_stable.py)::

    In [1]: run -t uniq_stable

    IPython CPU timings (estimated):
      User  :    0.19597 s.
      System:        0.0 s.

    In [2]: run -t -N5 uniq_stable

    IPython CPU timings (estimated):
    Total runs performed: 5
      Times :      Total       Per run
      User  :   0.910862 s,  0.1821724 s.
      System:        0.0 s,        0.0 s.

-d
  run your program under the control of pdb, the Python debugger.
```

```
  This allows you to execute your program step by step, watch variables,
  etc.  Internally, what IPython does is similar to calling::

      pdb.run('execfile("YOURFILENAME")')

  with a breakpoint set on line 1 of your file.  You can change the line
  number for this automatic breakpoint to be <N> by using the -bN option
  (where N must be an integer). For example::

      %run -d -b40 myscript

  will set the first breakpoint at line 40 in myscript.py.  Note that
  the first breakpoint must be set on a line which actually does
  something (not a comment or docstring) for it to stop execution.

  Or you can specify a breakpoint in a different file::

      %run -d -b myotherfile.py:20 myscript

  When the pdb debugger starts, you will see a (Pdb) prompt.  You must
  first enter 'c' (without quotes) to start execution up to the first
  breakpoint.

  Entering 'help' gives information about the use of the debugger.  You
  can easily see pdb's full documentation with "import pdb;pdb.help()"
  at a prompt.

-p
  run program under the control of the Python profiler module (which
  prints a detailed report of execution times, function calls, etc).

  You can pass other options after -p which affect the behavior of the
  profiler itself. See the docs for %prun for details.

  In this mode, the program's variables do NOT propagate back to the
  IPython interactive namespace (because they remain in the namespace
  where the profiler executes them).

  Internally this triggers a call to %prun, see its documentation for
  details on the options available specifically for profiling.

There is one special usage for which the text above doesn't apply:
if the filename ends with .ipy[nb], the file is run as ipython script,
just as if the commands were written on IPython prompt.

-m
  specify module name to load instead of script path. Similar to
  the -m option for the python interpreter. Use this option last if you
  want to combine with other %run options. Unlike the python interpreter
  only source modules are allowed no .pyc or .pyo files.
  For example::

      %run -m example

  will run the example module.
```

```
    -G
      disable shell-like glob expansion of arguments.
%save:
    Save a set of lines or a macro to a given filename.

    Usage:
      %save [options] filename [history]

    Options:

      -r: use 'raw' input.  By default, the 'processed' history is used,
      so that magics are loaded in their transformed version to valid
      Python.  If this option is given, the raw input as typed as the
      command line is used instead.

      -f: force overwrite.  If file exists, %save will prompt for overwrite
      unless -f is given.

      -a: append to the file instead of overwriting it.

    The history argument uses the same syntax as %history for input ranges,
    then saves the lines to the filename you specify.

    If no ranges are specified, saves history of the current session up to
    this point.

    It adds a '.py' extension to the file if you don't do so yourself, and
    it asks for confirmation before overwriting existing files.

    If `-r` option is used, the default extension is `.ipy`.
%sc:
    Shell capture - run shell command and capture output (DEPRECATED use !).

    DEPRECATED. Suboptimal, retained for backwards compatibility.

    You should use the form 'var = !command' instead. Example:

     "%sc -l myfiles = ls ~" should now be written as

     "myfiles = !ls ~"

    myfiles.s, myfiles.l and myfiles.n still apply as documented
    below.

    --
    %sc [options] varname=command

    IPython will run the given command using commands.getoutput(), and
    will then update the user's interactive namespace with a variable
    called varname, containing the value of the call.  Your command can
    contain shell wildcards, pipes, etc.

    The '=' sign in the syntax is mandatory, and the variable name you
    supply must follow Python's standard conventions for valid names.

    (A special format without variable name exists for internal use)
```

```
Options:

  -l: list output.  Split the output on newlines into a list before
  assigning it to the given variable.  By default the output is stored
  as a single string.

  -v: verbose.  Print the contents of the variable.

In most cases you should not need to split as a list, because the
returned value is a special type of string which can automatically
provide its contents either as a list (split on newlines) or as a
space-separated string.  These are convenient, respectively, either
for sequential processing or to be passed to a shell command.

For example::

    # Capture into variable a
    In [1]: sc a=ls *py

    # a is a string with embedded newlines
    In [2]: a
    Out[2]: 'setup.py\nwin32_manual_post_install.py'

    # which can be seen as a list:
    In [3]: a.l
    Out[3]: ['setup.py', 'win32_manual_post_install.py']

    # or as a whitespace-separated string:
    In [4]: a.s
    Out[4]: 'setup.py win32_manual_post_install.py'

    # a.s is useful to pass as a single command line:
    In [5]: !wc -l $a.s
      146 setup.py
      130 win32_manual_post_install.py
      276 total

    # while the list form is useful to loop over:
    In [6]: for f in a.l:
       ...:      !wc -l $f
       ...:
    146 setup.py
    130 win32_manual_post_install.py

Similarly, the lists returned by the -l option are also special, in
the sense that you can equally invoke the .s attribute on them to
automatically get a whitespace-separated string from their contents::

    In [7]: sc -l b=ls *py

    In [8]: b
    Out[8]: ['setup.py', 'win32_manual_post_install.py']

    In [9]: b.s
    Out[9]: 'setup.py win32_manual_post_install.py'
```

```
    In summary, both the lists and strings used for output capture have
    the following special attributes::

        .l (or .list) : value as list.
        .n (or .nlstr): value as newline-separated string.
        .s (or .spstr): value as space-separated string.
%set_env:
    Set environment variables.  Assumptions are that either "val" is a
    name in the user namespace, or val is something that evaluates to a
    string.

    Usage:
      :``%set_env var val``: set value for var
      :``%set_env var=val``: set value for var
      :``%set_env var=$val``: set value for var, using python expansion if p
ossible
%store:
    Lightweight persistence for python variables.

    Example::

      In [1]: l = ['hello',10,'world']
      In [2]: %store l
      Stored 'l' (list)
      In [3]: exit

      (IPython session is closed and started again...)

      ville@badger:~$ ipython
      In [1]: l
      NameError: name 'l' is not defined
      In [2]: %store -r
      In [3]: l
      Out[3]: ['hello', 10, 'world']

    Usage:

    * ``%store``          - Show list of all variables and their current
                            values
    * ``%store spam bar`` - Store the *current* value of the variables spam
                            and bar to disk
    * ``%store -d spam``  - Remove the variable and its value from storage
    * ``%store -z``       - Remove all variables from storage
    * ``%store -r``       - Refresh all variables, aliases and directory his
tory
                            from store (overwrite current vals)
    * ``%store -r spam bar`` - Refresh specified variables and aliases from
store
                               (delete current val)
    * ``%store foo >a.txt``  - Store value of foo to new file a.txt
    * ``%store foo >>a.txt`` - Append value of foo to file a.txt

    It should be noted that if you change the value of a variable, you
    need to %store it again if you want to persist the new value.
```

```
    Note also that the variables will need to be pickleable; most basic
    python types can be safely %store'd.

    Also aliases can be %store'd across sessions.
    To remove an alias from the storage, use the %unalias magic.
%sx:
    Shell execute - run shell command and capture output (!! is short-hand).

    %sx command

    IPython will run the given command using commands.getoutput(), and
    return the result formatted as a list (split on '\n').  Since the
    output is _returned_, it will be stored in ipython's regular output
    cache Out[N] and in the '_N' automatic variables.

    Notes:

    1) If an input line begins with '!!', then %sx is automatically
    invoked.  That is, while::

      !ls

    causes ipython to simply issue system('ls'), typing::

      !!ls

    is a shorthand equivalent to::

      %sx ls

    2) %sx differs from %sc in that %sx automatically splits into a list,
    like '%sc -l'.  The reason for this is to make it as easy as possible
    to process line-oriented shell output via further python commands.
    %sc is meant to provide much finer control, but requires more
    typing.

    3) Just like %sc -l, this is a list with special attributes:
    ::

      .l (or .list) : value as list.
      .n (or .nlstr): value as newline-separated string.
      .s (or .spstr): value as whitespace-separated string.

    This is very useful when trying to use such lists as arguments to
    system commands.
%system:
    Shell execute - run shell command and capture output (!! is short-hand).

    %sx command

    IPython will run the given command using commands.getoutput(), and
    return the result formatted as a list (split on '\n').  Since the
    output is _returned_, it will be stored in ipython's regular output
    cache Out[N] and in the '_N' automatic variables.

    Notes:
```

```
    1) If an input line begins with '!!', then %sx is automatically
    invoked.  That is, while::

      !ls

    causes ipython to simply issue system('ls'), typing::

      !!ls

    is a shorthand equivalent to::

      %sx ls

    2) %sx differs from %sc in that %sx automatically splits into a list,
    like '%sc -l'.  The reason for this is to make it as easy as possible
    to process line-oriented shell output via further python commands.
    %sc is meant to provide much finer control, but requires more
    typing.

    3) Just like %sc -l, this is a list with special attributes:
    ::

      .l (or .list) : value as list.
      .n (or .nlstr): value as newline-separated string.
      .s (or .spstr): value as whitespace-separated string.

    This is very useful when trying to use such lists as arguments to
    system commands.
%tb:
    Print the last traceback.

    Optionally, specify an exception reporting mode, tuning the
    verbosity of the traceback. By default the currently-active exception
    mode is used. See %xmode for changing exception reporting modes.

    Valid modes: Plain, Context, Verbose, and Minimal.
%time:
    Time execution of a Python statement or expression.

    The CPU and wall clock times are printed, and the value of the
    expression (if any) is returned.  Note that under Win32, system time
    is always reported as 0, since it can not be measured.

    This function can be used both as a line and cell magic:

    - In line mode you can time a single-line statement (though multiple
      ones can be chained with using semicolons).

    - In cell mode, you can time the cell body (a directly
      following statement raises an error).

    This function provides very basic timing functionality.  Use the timeit
    magic for more control over the measurement.

    .. versionchanged:: 7.3
```

```
    User variables are no longer expanded,
    the magic line is always left unmodified.

Examples
--------
::

  In [1]: %time 2**128
  CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
  Wall time: 0.00
  Out[1]: 340282366920938463463374607431768211456L

  In [2]: n = 1000000

  In [3]: %time sum(range(n))
  CPU times: user 1.20 s, sys: 0.05 s, total: 1.25 s
  Wall time: 1.37
  Out[3]: 499999500000L

  In [4]: %time print('hello world')
  hello world
  CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
  Wall time: 0.00

.. note::
    The time needed by Python to compile the given expression will be
    reported if it is more than 0.1s.

    In the example below, the actual exponentiation is done by Python
    at compilation time, so while the expression can take a noticeable
    amount of time to compute, that time is purely due to the
    compilation::

        In [5]: %time 3**9999;
        CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
        Wall time: 0.00 s

        In [6]: %time 3**999999;
        CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
        Wall time: 0.00 s
        Compiler : 0.78 s
%timeit:
    Time execution of a Python statement or expression

    **Usage, in line mode:**

      %timeit [-n<N> -r<R> [-t|-c] -q -p<P> -o] statement

    **or in cell mode:**

      %%timeit [-n<N> -r<R> [-t|-c] -q -p<P> -o] setup_code

      code

      code...
```

```
    Time execution of a Python statement or expression using the timeit
    module.  This function can be used both as a line and cell magic:

    - In line mode you can time a single-line statement (though multiple
      ones can be chained with using semicolons).

    - In cell mode, the statement in the first line is used as setup code
      (executed but not timed) and the body of the cell is timed.  The cell
      body has access to any variables created in the setup code.

    Options:

    -n<N>: execute the given statement <N> times in a loop. If <N> is not
    provided, <N> is determined so as to get sufficient accuracy.

    -r<R>: number of repeats <R>, each consisting of <N> loops, and take the
    average result.
    Default: 7

    -t: use time.time to measure the time, which is the default on Unix.
    This function measures wall time.

    -c: use time.clock to measure the time, which is the default on
    Windows and measures wall time. On Unix, resource.getrusage is used
    instead and returns the CPU user time.

    -p<P>: use a precision of <P> digits to display the timing result.
    Default: 3

    -q: Quiet, do not print result.

    -o: return a TimeitResult that can be stored in a variable to inspect
    the result in more details.

    .. versionchanged:: 7.3
        User variables are no longer expanded,
        the magic line is always left unmodified.

    Examples
    --------
    ::

      In [1]: %timeit pass
      8.26 ns ± 0.12 ns per loop (mean ± std. dev. of 7 runs, 100000000 loop
s each)

      In [2]: u = None

      In [3]: %timeit u is None
      29.9 ns ± 0.643 ns per loop (mean ± std. dev. of 7 runs, 10000000 loop
s each)

      In [4]: %timeit -r 4 u == None

      In [5]: import time
```

```
      In [6]: %timeit -n1 time.sleep(2)

    The times reported by %timeit will be slightly higher than those
    reported by the timeit.py script when variables are accessed. This is
    due to the fact that %timeit executes the statement in the namespace
    of the shell, compared with timeit.py, which uses a single setup
    statement to import function or create variables. Generally, the bias
    does not matter as long as results from timeit.py are not mixed with
    those from %timeit.
%unalias:
    Remove an alias
%unload_ext:
    Unload an IPython extension by its module name.

    Not all extensions can be unloaded, only those which define an
    ``unload_ipython_extension`` function.
%uv:
    Run the uv package manager within the current kernel.

    Usage:
      %uv pip install [pkgs]
%who:
    Print all interactive variables, with some minimal formatting.

    If any arguments are given, only variables whose type matches one of
    these are printed.  For example::

      %who function str

    will only list functions and strings, excluding all other types of
    variables.  To find the proper type names, simply use type(var) at a
    command line to see how python prints type names.  For example:

    ::

      In [1]: type('hello')
      Out[1]: <type 'str'>

    indicates that the type name for strings is 'str'.

    ``%who`` always excludes executed names loaded through your configuratio
n
    file and things which are internal to IPython.

    This is deliberate, as typically you may load many modules and the
    purpose of %who is to show you only what you've manually defined.

    Examples
    --------

    Define two variables and list them with who::

      In [1]: alpha = 123

      In [2]: beta = 'test'
```

```
      In [3]: %who
      alpha   beta

      In [4]: %who int
      alpha

      In [5]: %who str
      beta
%who_ls:
    Return a sorted list of all interactive variables.

    If arguments are given, only variables of types matching these
    arguments are returned.

    Examples
    --------
    Define two variables and list them with who_ls::

      In [1]: alpha = 123

      In [2]: beta = 'test'

      In [3]: %who_ls
      Out[3]: ['alpha', 'beta']

      In [4]: %who_ls int
      Out[4]: ['alpha']

      In [5]: %who_ls str
      Out[5]: ['beta']
%whos:
    Like %who, but gives some extra information about each variable.

    The same type filtering of %who can be applied here.

    For all variables, the type is printed. Additionally it prints:

      - For {},[],(): their length.

      - For numpy arrays, a summary with shape, number of
        elements, typecode and size in memory.

      - Everything else: a string representation, snipping their middle if
        too long.

    Examples
    --------
    Define two variables and list them with whos::

      In [1]: alpha = 123

      In [2]: beta = 'test'

      In [3]: %whos
      Variable   Type        Data/Info
      ----------------------------------
```

```
      alpha      int         123
      beta       str         test
%xdel:
    Delete a variable, trying to clear it from anywhere that
    IPython's machinery has references to it. By default, this uses
    the identity of the named object in the user namespace to remove
    references held under other names. The object is also removed
    from the output history.

    Options
      -n : Delete the specified name from all namespaces, without
      checking their identity.
%xmode:
    Switch modes for the exception handlers.

    Valid modes: Plain, Context, Verbose, and Minimal.

    If called without arguments, acts as a toggle.

    When in verbose mode the value `--show` (and `--hide`)
    will respectively show (or hide) frames with ``__tracebackhide__ =
    True`` value set.
%%!:
    Shell execute - run shell command and capture output (!! is short-hand).

    %sx command

    IPython will run the given command using commands.getoutput(), and
    return the result formatted as a list (split on '\n').  Since the
    output is _returned_, it will be stored in ipython's regular output
    cache Out[N] and in the '_N' automatic variables.

    Notes:

    1) If an input line begins with '!!', then %sx is automatically
    invoked.  That is, while::

      !ls

    causes ipython to simply issue system('ls'), typing::

      !!ls

    is a shorthand equivalent to::

      %sx ls

    2) %sx differs from %sc in that %sx automatically splits into a list,
    like '%sc -l'.  The reason for this is to make it as easy as possible
    to process line-oriented shell output via further python commands.
    %sc is meant to provide much finer control, but requires more
    typing.

    3) Just like %sc -l, this is a list with special attributes:
    ::
```

```
      .l (or .list) : value as list.
      .n (or .nlstr): value as newline-separated string.
      .s (or .spstr): value as whitespace-separated string.

    This is very useful when trying to use such lists as arguments to
    system commands.
%%HTML:
    Alias for `%%html`.
%%SVG:
    Alias for `%%svg`.
%%bash:
    %%bash script magic

    Run cells with bash in a subprocess.

    This is a shortcut for `%%script bash`
%%capture:
    ::

      %capture [--no-stderr] [--no-stdout] [--no-display] [output]

    run the cell, capturing stdout, stderr, and IPython's rich display() cal
ls.

    positional arguments:
      output        The name of the variable in which to store output. This
is a
                    utils.io.CapturedIO object with stdout/err attributes fo
r the
                    text of the captured output. CapturedOutput also has a s
how()
                    method for displaying the output, and __call__ as well,
so you
                    can use that to quickly display the output. If unspecifi
ed,
                    captured output is discarded.

    options:
      --no-stderr   Don't capture stderr.
      --no-stdout   Don't capture stdout.
      --no-display  Don't capture IPython's rich display.
%%code_wrap:
    ::

      %code_wrap [--remove] [--list] [--list-all] [name]

    Simple magic to quickly define a code transformer for all IPython's futu
re input.

    ``__code__`` and ``__ret__`` are special variable that represent the cod
e to run
    and the value of the last expression of ``__code__`` respectively.

    Examples
    --------
```

```
        .. ipython::

            In [1]: %%code_wrap before_after
               ...: print('before')
               ...: __code__
               ...: print('after')
               ...: __ret__

            In [2]: 1
            before
            after
            Out[2]: 1

            In [3]: %code_wrap --list
            before_after

            In [4]: %code_wrap --list-all
            before_after :
                print('before')
                __code__
                print('after')
                __ret__

            In [5]: %code_wrap --remove before_after

        positional arguments:
          name

        options:
          --remove    remove the current transformer
          --list      list existing transformers name
          --list-all  list existing transformers name and code template
    %%debug:
        ::

          %debug [--breakpoint FILE:LINE] [statement ...]

        Activate the interactive debugger.

        This magic command support two ways of activating debugger.
        One is to activate debugger before executing code.  This way, you
        can set a break point, to step through the code from the point.
        You can use this mode by giving statements to execute and optionally
        a breakpoint.

        The other one is to activate debugger in post-mortem mode.  You can
        activate this mode simply running %debug without any argument.
        If an exception has just occurred, this lets you inspect its stack
        frames interactively.  Note that this will always work only on the last
        traceback that occurred, so you must call this quickly after an
        exception that you wish to inspect has fired, because if another one
        occurs, it clobbers the previous one.

        If you want IPython to automatically do this on every exception, see
        the %pdb magic for more details.
```

```
    .. versionchanged:: 7.3
        When running code, user variables are no longer expanded,
        the magic line is always left unmodified.

    positional arguments:
      statement             Code to run in debugger. You can omit this in ce
ll
                            magic mode.

    options:
      --breakpoint <FILE:LINE>, -b <FILE:LINE>
                            Set break point at LINE in FILE.
%%file:
    Alias for `%%writefile`.
%%html:
    ::

      %html [--isolated]

    Render the cell as a block of HTML

    options:
      --isolated  Annotate the cell as 'isolated'. Isolated cells are render
ed
                  inside their own <iframe> tag
%%javascript:
    Run the cell block of Javascript code

    Starting with IPython 8.0 %%javascript is pending deprecation to be repl
aced
    by a more flexible system

    Please See https://github.com/ipython/ipython/issues/13376
%%js:
    Run the cell block of Javascript code

    Alias of `%%javascript`

    Starting with IPython 8.0 %%javascript is pending deprecation to be repl
aced
    by a more flexible system

    Please See https://github.com/ipython/ipython/issues/13376
%%latex:
    Render the cell as a block of LaTeX

    The subset of LaTeX which is supported depends on the implementation in
    the client.  In the Jupyter Notebook, this magic only renders the subset
    of LaTeX defined by MathJax
    [here](https://docs.mathjax.org/en/v2.5-latest/tex.html).
%%markdown:
    Render the cell as Markdown text block
%%perl:
    %%perl script magic

    Run cells with perl in a subprocess.
```

```
    This is a shortcut for `%%script perl`
%%prun:
    Run a statement through the python code profiler.

    **Usage, in line mode:**

      %prun [options] statement

    **Usage, in cell mode:**

      %%prun [options] [statement]

      code...

      code...

    In cell mode, the additional code lines are appended to the (possibly
    empty) statement in the first line.  Cell mode allows you to easily
    profile multiline blocks without having to put them in a separate
    function.

    The given statement (which doesn't require quote marks) is run via the
    python profiler in a manner similar to the profile.run() function.
    Namespaces are internally managed to work correctly; profile.run
    cannot be used in IPython because it makes certain assumptions about
    namespaces which do not hold under IPython.

    Options:

    -l <limit>
      you can place restrictions on what or how much of the
      profile gets printed. The limit value can be:

         * A string: only information for function names containing this str
ing
           is printed.

         * An integer: only these many lines are printed.

         * A float (between 0 and 1): this fraction of the report is printed
           (for example, use a limit of 0.4 to see the topmost 40% only).

      You can combine several limits with repeated use of the option. For
      example, ``-l __init__ -l 5`` will print only the topmost 5 lines of
      information about class constructors.

    -r
      return the pstats.Stats object generated by the profiling. This
      object has all the information about the profile in it, and you can
      later use it for further analysis or in other functions.

    -s <key>
      sort profile by given key. You can provide more than one key
      by using the option several times: '-s key1 -s key2 -s key3...'. The
      default sorting key is 'time'.
```

```
        The following is copied verbatim from the profile documentation
        referenced below:

        When more than one key is provided, additional keys are used as
        secondary criteria when the there is equality in all keys selected
        before them.

        Abbreviations can be used for any key names, as long as the
        abbreviation is unambiguous.  The following are the keys currently
        defined:

        ============  =====================
        Valid Arg     Meaning
        ============  =====================
        "calls"       call count
        "cumulative"  cumulative time
        "file"        file name
        "module"      file name
        "pcalls"      primitive call count
        "line"        line number
        "name"        function name
        "nfl"         name/file/line
        "stdname"     standard name
        "time"        internal time
        ============  =====================

        Note that all sorts on statistics are in descending order (placing
        most time consuming items first), where as name, file, and line number
        searches are in ascending order (i.e., alphabetical). The subtle
        distinction between "nfl" and "stdname" is that the standard name is a
        sort of the name as printed, which means that the embedded line
        numbers get compared in an odd way.  For example, lines 3, 20, and 40
        would (if the file names were the same) appear in the string order
        "20" "3" and "40".  In contrast, "nfl" does a numeric compare of the
        line numbers.  In fact, sort_stats("nfl") is the same as
        sort_stats("name", "file", "line").

    -T <filename>
        save profile results as shown on screen to a text
        file. The profile is still shown on screen.

    -D <filename>
        save (via dump_stats) profile statistics to given
        filename. This data is in a format understood by the pstats module, an
d
        is generated by a call to the dump_stats() method of profile
        objects. The profile is still shown on screen.

    -q
        suppress output to the pager.  Best used with -T and/or -D above.

    If you want to run complete programs under the profiler's control, use
    ``%run -p [prof_opts] filename.py [args to program]`` where prof_opts
    contains profiler specific options as described here.
```

```
    You can read the complete documentation for the profile module with::

      In [1]: import profile; profile.help()

    .. versionchanged:: 7.3
        User variables are no longer expanded,
        the magic line is always left unmodified.
%%pypy:
    %%pypy script magic

    Run cells with pypy in a subprocess.

    This is a shortcut for `%%script pypy`
%%python:
    %%python script magic

    Run cells with python in a subprocess.

    This is a shortcut for `%%script python`
%%python2:
    %%python2 script magic

    Run cells with python2 in a subprocess.

    This is a shortcut for `%%script python2`
%%python3:
    %%python3 script magic

    Run cells with python3 in a subprocess.

    This is a shortcut for `%%script python3`
%%ruby:
    %%ruby script magic

    Run cells with ruby in a subprocess.

    This is a shortcut for `%%script ruby`
%%script:
    ::

      %shebang [--no-raise-error] [--proc PROC] [--bg] [--err ERR] [--out OU
T]

    Run a cell via a shell command

    The `%%script` line is like the #! line of script,
    specifying a program (bash, perl, ruby, etc.) with which to run.

    The rest of the cell is run by that program.

    Examples
    --------
    ::

        In [1]: %%script bash
           ...: for i in 1 2 3; do
```

```
           ...:   echo $i
           ...: done
          1
          2
          3

    options:
      --no-raise-error  Whether you should raise an error message in additio
n to a
                        stream on stderr if you get a nonzero exit code.
      --proc PROC       The variable in which to store Popen instance. This
is
                        used only when --bg option is given.
      --bg              Whether to run the script in the background. If give
n, the
                        only way to see the output of the command is with
                        --out/err.
      --err ERR         The variable in which to store stderr from the scrip
t. If
                        the script is backgrounded, this will be the stderr
                        *pipe*, instead of the stderr text itself and will n
ot be
                        autoclosed.
      --out OUT         The variable in which to store stdout from the scrip
t. If
                        the script is backgrounded, this will be the stdout
                        *pipe*, instead of the stderr text itself and will n
ot be
                        auto closed.
%%sh:
    %%sh script magic

    Run cells with sh in a subprocess.

    This is a shortcut for `%%script sh`
%%svg:
    Render the cell as an SVG literal
%%sx:
    Shell execute - run shell command and capture output (!! is short-hand).

    %sx command

    IPython will run the given command using commands.getoutput(), and
    return the result formatted as a list (split on '\n').  Since the
    output is _returned_, it will be stored in ipython's regular output
    cache Out[N] and in the '_N' automatic variables.

    Notes:

    1) If an input line begins with '!!', then %sx is automatically
    invoked.  That is, while::

      !ls

    causes ipython to simply issue system('ls'), typing::
```

```
      !!ls

    is a shorthand equivalent to::

      %sx ls

    2) %sx differs from %sc in that %sx automatically splits into a list,
    like '%sc -l'.  The reason for this is to make it as easy as possible
    to process line-oriented shell output via further python commands.
    %sc is meant to provide much finer control, but requires more
    typing.

    3) Just like %sc -l, this is a list with special attributes:
    ::

      .l (or .list) : value as list.
      .n (or .nlstr): value as newline-separated string.
      .s (or .spstr): value as whitespace-separated string.

    This is very useful when trying to use such lists as arguments to
    system commands.
%%system:
    Shell execute - run shell command and capture output (!! is short-hand).

    %sx command

    IPython will run the given command using commands.getoutput(), and
    return the result formatted as a list (split on '\n').  Since the
    output is _returned_, it will be stored in ipython's regular output
    cache Out[N] and in the '_N' automatic variables.

    Notes:

    1) If an input line begins with '!!', then %sx is automatically
    invoked.  That is, while::

      !ls

    causes ipython to simply issue system('ls'), typing::

      !!ls

    is a shorthand equivalent to::

      %sx ls

    2) %sx differs from %sc in that %sx automatically splits into a list,
    like '%sc -l'.  The reason for this is to make it as easy as possible
    to process line-oriented shell output via further python commands.
    %sc is meant to provide much finer control, but requires more
    typing.

    3) Just like %sc -l, this is a list with special attributes:
    ::

      .l (or .list) : value as list.
```

```
      .n (or .nlstr): value as newline-separated string.
      .s (or .spstr): value as whitespace-separated string.

    This is very useful when trying to use such lists as arguments to
    system commands.
%%time:
    Time execution of a Python statement or expression.

    The CPU and wall clock times are printed, and the value of the
    expression (if any) is returned.  Note that under Win32, system time
    is always reported as 0, since it can not be measured.

    This function can be used both as a line and cell magic:

    - In line mode you can time a single-line statement (though multiple
      ones can be chained with using semicolons).

    - In cell mode, you can time the cell body (a directly
      following statement raises an error).

    This function provides very basic timing functionality.  Use the timeit
    magic for more control over the measurement.

    .. versionchanged:: 7.3
        User variables are no longer expanded,
        the magic line is always left unmodified.

    Examples
    --------
    ::

      In [1]: %time 2**128
      CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
      Wall time: 0.00
      Out[1]: 340282366920938463463374607431768211456L

      In [2]: n = 1000000

      In [3]: %time sum(range(n))
      CPU times: user 1.20 s, sys: 0.05 s, total: 1.25 s
      Wall time: 1.37
      Out[3]: 499999500000L

      In [4]: %time print('hello world')
      hello world
      CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
      Wall time: 0.00

    .. note::
        The time needed by Python to compile the given expression will be
        reported if it is more than 0.1s.

        In the example below, the actual exponentiation is done by Python
        at compilation time, so while the expression can take a noticeable
        amount of time to compute, that time is purely due to the
        compilation::
```

```
In [5]: %time 3**9999;
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.00 s

In [6]: %time 3**999999;
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.00 s
Compiler : 0.78 s
```

```
%%timeit:
    Time execution of a Python statement or expression

    **Usage, in line mode:**

      %timeit [-n<N> -r<R> [-t|-c] -q -p<P> -o] statement

    **or in cell mode:**

      %%timeit [-n<N> -r<R> [-t|-c] -q -p<P> -o] setup_code

      code

      code...

    Time execution of a Python statement or expression using the timeit
    module.  This function can be used both as a line and cell magic:

    - In line mode you can time a single-line statement (though multiple
      ones can be chained with using semicolons).

    - In cell mode, the statement in the first line is used as setup code
      (executed but not timed) and the body of the cell is timed.  The cell
      body has access to any variables created in the setup code.

    Options:

    -n<N>: execute the given statement <N> times in a loop. If <N> is not
    provided, <N> is determined so as to get sufficient accuracy.

    -r<R>: number of repeats <R>, each consisting of <N> loops, and take the
    average result.
    Default: 7

    -t: use time.time to measure the time, which is the default on Unix.
    This function measures wall time.

    -c: use time.clock to measure the time, which is the default on
    Windows and measures wall time. On Unix, resource.getrusage is used
    instead and returns the CPU user time.

    -p<P>: use a precision of <P> digits to display the timing result.
    Default: 3

    -q: Quiet, do not print result.

    -o: return a TimeitResult that can be stored in a variable to inspect
```

```
    the result in more details.

    .. versionchanged:: 7.3
        User variables are no longer expanded,
        the magic line is always left unmodified.

    Examples
    --------
    ::

      In [1]: %timeit pass
      8.26 ns ± 0.12 ns per loop (mean ± std. dev. of 7 runs, 100000000 loop
s each)

      In [2]: u = None

      In [3]: %timeit u is None
      29.9 ns ± 0.643 ns per loop (mean ± std. dev. of 7 runs, 10000000 loop
s each)

      In [4]: %timeit -r 4 u == None

      In [5]: import time

      In [6]: %timeit -n1 time.sleep(2)

    The times reported by %timeit will be slightly higher than those
    reported by the timeit.py script when variables are accessed. This is
    due to the fact that %timeit executes the statement in the namespace
    of the shell, compared with timeit.py, which uses a single setup
    statement to import function or create variables. Generally, the bias
    does not matter as long as results from timeit.py are not mixed with
    those from %timeit.
%%writefile:
    ::

      %writefile [-a] filename

    Write the contents of the cell to a file.

    The file will be overwritten unless the -a (--append) flag is specified.

    positional arguments:
      filename      file to write

    options:
      -a, --append  Append contents of the cell to an existing file. The fil
e will
                    be created if it does not exist.

Summary of magic functions (from %lsmagic):
Available line magics:
%alias  %alias_magic  %autoawait  %autocall  %automagic  %autosave  %bookmar
k  %cat  %cd  %clear  %code_wrap  %colors  %conda  %config  %connect_info  %
cp  %debug  %dhist  %dirs  %doctest_mode  %ed  %edit  %env  %gui  %hist  %hi
story  %killbgscripts  %ldir  %less  %lf  %lk  %ll  %load  %load_ext  %loadp
```

```
y  %logoff  %logon  %logstart  %logstate  %logstop  %ls  %lsmagic  %lx  %mac
ro  %magic  %mamba  %man  %matplotlib  %micromamba  %mkdir  %more  %mv  %not
ebook  %page  %pastebin  %pdb  %pdef  %pdoc  %pfile  %pinfo  %pinfo2  %pip
%popd  %pprint  %precision  %prun  %psearch  %psource  %pushd  %pwd  %pycat
%pylab  %qtconsole  %quickref  %recall  %rehashx  %reload_ext  %rep  %rerun
%reset  %reset_selective  %rm  %rmdir  %run  %save  %sc  %set_env  %store  %
sx  %system  %tb  %time  %timeit  %unalias  %unload_ext  %uv  %who  %who_ls
%whos  %xdel  %xmode

Available cell magics:
%%!  %%HTML  %%SVG  %%bash  %%capture  %%code_wrap  %%debug  %%file  %%html
%%javascript  %%js  %%latex  %%markdown  %%perl  %%prun  %%pypy  %%python  %
%python2  %%python3  %%ruby  %%script  %%sh  %%svg  %%sx  %%system  %%time
%%timeit  %%writefile

Automagic is ON, % prefix IS NOT needed for line magics.
```

In [5]:
```
import numpy as np

a = np.random.rand(100)
%timeit a.sum()
```

```
548 ns ± 2.94 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
```

Question 4 (1 pt)

It's useful to know where python is located on your computer and what version of python is installed; try out the following terminal commands: `! which python` and `! python --version`

Your solution:

In [6]:
```
! which python
```

```
/Users/ameygaikwad/anaconda3/bin/python
```

In [9]:
```
! python --version
```

```
Python 3.11.4
```

Question 5 (1 pt)

Jupyter also supports LaTeX in markdown (double-click this cell to see the markdown syntax). For inline math, surround expressions with single dollar signs like this: $e^{i\pi} + 1 = 0$ $\rightarrow e^{i\pi} + 1 = 0$

Double dollar signs around expressions produce equations: $$ e^x=\sum_{i=0}^\infty \frac{1}{i!}x^i $$ becomes

$$e^x = \sum_{i=0}^{\infty} \frac{1}{i!} x^i$$

Type out any LaTeX equation to test it out in the cell below:

$$E = mc^2$$

Python Review

Question 6 (2 pts)

First, define a function called `mult` that accepts two variables, multiplies the variables, and returns the product.

Your solution:

In [10]:
```
def mult(x,y):
    return x*y
```

Next, call `mult` and pass it 5 and 3. Assign the output to a variable named `product`.

Your solution:

In [11]:
```
product=mult(5,3)
print(product)
```

```
15
```

Modify `mult` so that, instead of returning the product of two integer or float inputs, it returns the dot product between the vectors defined by two lists or NumPy arrays (*hint:* look at the documentation for the linear algebra methods in NumPy.) With your new function, assign the output of the product between arrays `[1,2,3]` and `[4,5,6]` to a variable valled `dotprod`, then print `dotprod`.

Your solution:

In [12]:
```
#Brute Force
def mult(arr1,arr2):
    l=len(arr1)
    sum=0
    for i in range(l):
        sum=sum+arr1[i]*arr2[i]
    return sum
```

In [13]:
```
import numpy as np
def mult(arr1,arr2):
    return np.dot(arr1,arr2)
```

In [14]:
```
dotprod=mult([1,2,3],[4,5,6])
print(dotprod)
```

32

Lists and Arrays

Array algebra is the core of lots of machine learning, and Numpy is the package we'll use to store and operate on arrays. *Note* that one important challenge is minimizing the computational cost of array algebra.

Question 7 (2 pts)

In the cell below, assign a list containing integers spanning the interval between 0 and 9 to a variable named `l`. Use a NumPy method to generate the list; don't type in each integer. *Note* that, if the method you use generates an array, you'll need to convert the array to a list.

Your solution:

In [15]:
```
import numpy as np

l=np.arange(0,10).tolist()
```

Run the cell below to verify that `l` is the desired list.

In [16]:
```
assert l == [0,1,2,3,4,5,6,7,8,9]
```

Using list comprehension, generate the same list and assign it to a variable called `l2`.

Your solution:

In [17]:
```
l2=[i for i in range(10)]
```

Run the cell below to verify that `l2` is the desired list.

In [18]:
```
assert l2 == [0,1,2,3,4,5,6,7,8,9]
```

Using the `np.array()` method, assign an array with the same elements as the list `l2` to a variable named `arr` (generate the array using `l2`.)

Your solution:

In [19]:
```
arr=np.array(l2)
```

Run the cell below to verify.

In [20]:
```
assert isinstance(arr, np.ndarray)
```

Question 8 (7 pts)

Unlike lists, arrays can have multiple dimensions of data. Non-vector tensors are the bulk of the arrays which appear in machine learning problems. Using a NumPy method, create a 3×5 array called `arr2` with 1 for every element.

Your solution:

In [21]:
```
arr2=np.ones((3,5))
```

Verify your solution:

In [22]:
```
assert_array_equal(arr2,np.ones((3,5)))
```

Another common task in ML is to create arrays filled with random numbers. There are lots of different NumPy methods for generating random numbers! Create a 3×3 array of random integers normally distributed on the interval `[0,10]`, and assign it to a variable called `arr3`.

Your solution:

In [23]:
```
arr3=np.random.normal(loc=5,scale=2,size=(3,3))
arr3=np.clip(arr3,0,10)
arr3=np.rint(arr3).astype(int)
```

Verify the shape of your array:

In [24]:
```
assert arr3.shape == (3,3)
```

Assign the `shape` attribute of `arr3` to a variable called `s`.

Your solution:

In [25]:
```
s=arr3.shape
```

Verify the type of `s`:

In [26]:
```
assert isinstance(s, tuple)
```

Assign the number of array dimensions of `arr3` to a variable called `d`. You can find the right method in this guide.

Your solution:

In [27]:
```
d=arr3.ndim
```

Using array indexing, print the element of `arr3` in the first row and first column and assign it to a variable called `x`.

Your solution:

In [28]:
```
x= arr3[0,0]
```

In [29]:
```
assert x == arr3[0,0]
```

Create a variable `e` with the value of the element in the row with the largest index and column with the largest index in `arr3`. *Note* that there is a way to index the last column or row that doesn't depend on the size of the array.

Your solution:

In [30]:
```
e= arr3[-1,-1]
```

In [31]:
```
assert e == arr3[-1,-1]
```

Since datasets are often large in ML contexts, we need to slice arrays and only use a portion of a dataset. One way to slice an array is using the command `array[start_index:stop_index]`. We can also specify a step size, `array[start_index:stop_index:step_size]`. Create a new variable called `y` which contains just the first two rows of `arr3`.

Your solution:

In [32]:
```
y=arr3[:2,:]
```

In [33]:
```
assert y.all() == arr3[:2].all()
```

Using NumPy's `arange` and `reshape` methods, create an array called `arr55` so that `print(arr55)` returns this:

```
[[1,2,3,4,5],   [6,7,8,9,10],   [11,12,13,14,15],
[16,17,18,19,20],   [21,22,23,24,25]]
```

Your solution:

In [34]:
```
arr55=np.arange(1,26).reshape(5,5)
```

In [35]:
```
assert arr55.all() == np.arange(1,26).reshape(5,5).all()
```

Broadcasting:

Broadcasting, defined by VanderPlas: a set of rules for applying binary ufuncs (e.g., addition, multiplication, etc.) on arrays of different sizes. For arrays of the same size, binary operations are performed on an element-by-element basis. Broadcasting allows

these operations to be performed on arrays of different sizes, for example adding a scalar to an array. Here's a brief demonstration of how it works:

Adding two arrays with different shapes:

First, define an array with shape (4,1).

```
a = np.array(         [[1.],          [2.],          [3.],          [4.]])
```

Note that the periods after the numbers tell Python that the elements should have *float* DataTypes rather than *integer*. Now define another array with shape (3,).

```
b = np.array([0., 1., 2.])
```

Now we operate on the arrays:

```
c = a + b
```

And find that

```
c = np.array([[1., 2., 3.],        [2., 3., 4.],        [3., 4., 5.],        [4., 5., 6.]])
```

Here's what happened inside the NumPy machinery:

NumPy aligned the shapes of a and b on the last axis and prepended 1s to the shape with fewer axes: a: 4 x 1 ---> a: 4 x 1 b: 3 ---> b: 1 x 3 Then checked that the sizes of the axes matched or were equal to 1:
a: 4 x 1
b: 1 x 3
`a` and `b` satisfied this criterion.
NumPy stretched both arrays on their 1-valued axes so that their shapes matched, then added them together. `a` was replicated 3 times in the second axis, while `b` was replicated 4 times in the first axis. This meant that the addition in the final step was

```
[[1., 1., 1.],           [[0., 1., 2.],    [2., 2., 2.],      +
[0., 1., 2.],    [3., 3., 3.],            [0., 1., 2.],    [4., 4., 4.]]            [0., 1., 2.]]
```

Addition was then carried out element-by-element, as you can verify by referring back to the output of the code cell above. This resulted in an output with shape 4 x 3.

Numpy's broadcasting rule

Broadcasting rules describe how values should be transmitted when the inputs to an operation do not match. In numpy, the broadcasting rule is very simple:

Prepend 1s to the smaller shape, check that the axes of both arrays have sizes that are equal or 1, then stretch the arrays in their size-1 axes.

A crucial aspect of this rule is that it does not require the input arrays have the same number of axes. Another consequence of it is that a broadcasting output will have the largest size of its inputs in each axis. Take the following multiplication as an example:

```
a: 3 x 7 x 1
b:     1 x 5
```

a * b: 3 x 7 x 5 You can see that the output shape is the maximum of the sizes in each axis.

NumPy's broadcasting rule also does not require that one of the arrays has to be bigger in all axes.

Broadcasting behavior also points to an efficient way to compute an outer product in NumPy:

In [36]:
```
a = np.array([-1., 0., 1.])
b = np.array([0., 1., 2., 3.])

a[:, np.newaxis] * b  # outer product ab^T, where a and b are column vectors
```

Out[36]:
```
array([[-0., -1., -2., -3.],
       [ 0.,  0.,  0.,  0.],
       [ 0.,  1.,  2.,  3.]])
```

The idea of numpy stretching the arrays in their size-1 axes is useful and is functionally correct. But this is not what numpy literally does behind the scenes, since that would be an inefficient use of memory. Instead, numpy carries out the operation by looping over singleton (size-1) dimensions.

Question 9 (2 pts)

To give you some practise with broadcasting, **try predicting the output shapes for the following operations**:

In [37]:
```
# Run this cell, which defines three arrays with different shapes
a = np.array([[1.], [2.], [3.]])
b = np.zeros(shape=[10, 1, 1])
c = np.ones(shape=[4])
d = np.array([1,2,3,4])
```

What are the shapes of `a`, `b`, `c`, and `d`? Define `d1` as the shape of `d`.

In [38]:
```
a1=(3,1)
b1=(10,1,1)
```

```
c1=(4,)
d1=(4,)
```

In [39]:
```
assert d1==(4,)
```

Multiply two arrays with differen shapes:

In [40]:
```
a = np.array([[[0.01], [0.1]],
              [[1.00], [10.]]])  # what shape is this? it's (2, 2, 1)
b = np.array([[[2., 2.]],
              [[3., 3.]]])       # what shape is this? it's (2, 1, 2)

c = a * b
```

What is the shape of `c`? Type the answer below and assign to `v`.

In [41]:
```
v=(2,2,2)
# last dimension 1 vs 2 -> broadcasts to 2 ; middle dimension 2 vs 1 -> broa
```

In [42]:
```
assert v == (2,2,2)
```

Pandas:

Pandas is a Python library built on Numpy. It defines Series and DataFrame objects, which are multi-dimensional arrays with row and column labels attached naturally. They are good with various data types and missing data. This is a link to the Pandas documentation.

In [43]:
```
import pandas as pd
```

A Pandas Series object is a one dimensional array of indexed data. Such an object can be created from a list in a number of ways, and so can DataFrames. We will try to introduce the various ways; they'll be confusing for now, but it gets easier with more exposure; you will start to see the different, equivalent ways of performing ML and data science tasks.

Question 10 (1 pt)

In [44]:
```
data = pd.Series([0.30, 0.5, 0.78, 1.0])
```

In the cells below, try data.values and data.index, and try to index the Series using `data[2]`.

In [45]:
```
data.values
```

Out[45]:
```
array([0.3 , 0.5 , 0.78, 1.  ])
```

In [46]:
```
data.index
```

Out[46]:
```
RangeIndex(start=0, stop=4, step=1)
```

In [47]:
```
data[2]
```

Out[47]:
```
0.78
```

DataFrames

Question 11 (8 pts)

You can think of a Pandas DataFrame object as a sequence of aligned Series objects (aligned in the sense that they share the same index.) These are some ways you can construct DataFrames: from a single Series object, from a list of dictionaries, from a dictionary of Series objects, from a 2-d numpy array, from a Numpy Structured array, or by importing some data (for example, a CSV file). If you know SQL, DataFrames are designed to allow for different joins of datasets, and lots of other functionalities.

Next, we are going to load a dataset and perform Pandas operations on the dataset.

We'll learn how to calculate statstics on datasets and how to renaming columns, add rows, drop columns, combine columns, create new columns out of existing ones, locate information with queries, filter, etc.

In [48]:
```
# (1 pt.)

import pandas as pd
import numpy as np
from sklearn.utils import Bunch
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
feature_names = np.array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE',
       'TAX', 'PTRATIO', 'B', 'LSTAT'])
boston = Bunch(data=data, target=target, feature_names=feature_names)

# in the space below, print the datatype of boston.
### BEGIN SOLUTION
print(type(boston))
### END SOLUTION
```

```
<class 'sklearn.utils._bunch.Bunch'>
```

Print the datatype of `boston` in the cell below.

In [49]:
```
print(type(boston))
#'boston' is a scikit learn Bunch -- a dictionary where I can access fields
```

```
<class 'sklearn.utils._bunch.Bunch'>
```

In [50]:
```
import sklearn as sk
assert isinstance(boston, sk.utils.Bunch) #bunch file has dicts of labels, s
```

(1pt.) **What is a bunch object? What is it made up of? Answer in the next cell.**

A Bunch object in scikit learn is a lightweight container -- a dictionary, used to bundle related pieces of data together and access them conveniently with dot notation. It is made up of key value pairs, like a dict. For instance it holds things like :

1. data : feature matrix (a numpy array, for instance)
2. target : labels and outputs
3. feature_names : names of columns and features

and other metadata

(1pt.) Transform just boston.data into a Pandas DataFrame (not the other items in the bunch) and name the dataframe `Xbos` . *Hint 1*: you want to assign the columns to be boston.feature_names. *Hint 2*: don't worry about the y value (the target,) we won't be using that for this exercise.

In [51]:
```
Xbos = pd.DataFrame(boston.data, columns = boston.feature_names)
print(Xbos.head())
```

```
      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0  0.00632  18.0   2.31   0.0  0.538  6.575  65.2  4.0900  1.0  296.0
1  0.02731   0.0   7.07   0.0  0.469  6.421  78.9  4.9671  2.0  242.0
2  0.02729   0.0   7.07   0.0  0.469  7.185  61.1  4.9671  2.0  242.0
3  0.03237   0.0   2.18   0.0  0.458  6.998  45.8  6.0622  3.0  222.0
4  0.06905   0.0   2.18   0.0  0.458  7.147  54.2  6.0622  3.0  222.0

   PTRATIO       B  LSTAT
0     15.3  396.90   4.98
1     17.8  396.90   9.14
2     17.8  392.83   4.03
3     18.7  394.63   2.94
4     18.7  396.90   5.33
```

In [52]:
```
from pandas.testing import assert_frame_equal
assert_frame_equal (Xbos, pd.DataFrame(boston.data, columns = boston.feature
```

(1pt.) Check out your dataframe! in the space below, try `Xbos.head()` , `Xbos['insert_col_name'].value_counts()` , `Xbos.values` , `Xbos.std()` , and `Xbos.mean()` . Refer back to the documentation in sklearn to understand the column values.

In [53]:
```
Xbos.head()
```

Out[53]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	3
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	3
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	3
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	3
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	3

In [54]:
```
Xbos['CHAS'].value_counts()
```

Out[54]:
```
CHAS
0.0    471
1.0     35
Name: count, dtype: int64
```

In [55]:
```
Xbos.values
```

Out[55]:
```
array([[6.3200e-03, 1.8000e+01, 2.3100e+00, ..., 1.5300e+01, 3.9690e+02,
        4.9800e+00],
       [2.7310e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9690e+02,
        9.1400e+00],
       [2.7290e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9283e+02,
        4.0300e+00],
       ...,
       [6.0760e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
        5.6400e+00],
       [1.0959e-01, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9345e+02,
        6.4800e+00],
       [4.7410e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
        7.8800e+00]])
```

In [56]:
```
Xbos.std()
```

Out[56]:
```
CRIM         8.601545
ZN          23.322453
INDUS        6.860353
CHAS         0.253994
NOX          0.115878
RM           0.702617
AGE         28.148861
DIS          2.105710
RAD          8.707259
TAX        168.537116
PTRATIO      2.164946
B           91.294864
LSTAT        7.141062
dtype: float64
```

In [57]:
```
Xbos.mean()
```

Out[57]:
```
CRIM         3.613524
ZN          11.363636
INDUS       11.136779
CHAS         0.069170
NOX          0.554695
RM           6.284634
AGE         68.574901
DIS          3.795043
RAD          9.549407
TAX        408.237154
PTRATIO     18.455534
B          356.674032
LSTAT       12.653063
dtype: float64
```

In [58]:
```
type(Xbos)
```

Out[58]:
```
pandas.core.frame.DataFrame
```

Now we are going to manipulate our dataframe Xbos. Keep in mind when querying and changing-- are you copying the dataframe or altering the actual dataframe, or getting a 'view' of a dataframe without actually altering it. When reading the Pandas documentation- think about ways to be careful here!

Let's start with selecting data with loc and iloc. (This is a common interview question for data science jobs/internships, to explain the difference)

This is a pretty decent set of examples to show the difference between loc and iloc: https://thispointer.com/select-rows-columns-by-name-or-index-in-dataframe-using-loc-iloc-python-pandas/

loc: loc is label-based, which means that we have to specify the name of the rows and columns that we need to filter out.

iloc: iloc is integer index-based. So here, we have to specify rows and columns by their integer index.

Starting with loc, try: Xbos.loc where CRIM >= 45. assign this to the variable cri. Answer in the cell below.

In [59]:
```
cri=Xbos.loc[Xbos["CRIM"]>=45]
cri.info
```

Out[59]:
```
<bound method DataFrame.info of         CRIM    ZN  INDUS  CHAS    NOX     R
M    AGE     DIS    RAD    TAX  \
380  88.9762  0.0   18.1   0.0  0.671  6.968   91.9  1.4165  24.0  666.0
405  67.9208  0.0   18.1   0.0  0.693  5.683  100.0  1.4254  24.0  666.0
410  51.1358  0.0   18.1   0.0  0.597  5.757  100.0  1.4130  24.0  666.0
414  45.7461  0.0   18.1   0.0  0.693  4.519  100.0  1.6582  24.0  666.0
418  73.5341  0.0   18.1   0.0  0.679  5.957  100.0  1.8026  24.0  666.0

     PTRATIO       B  LSTAT
380     20.2  396.90  17.21
405     20.2  384.97  22.98
410     20.2    2.60  10.11
414     20.2   88.27  36.98
418     20.2   16.45  20.62  >
```

In [60]:
```
assert_frame_equal (cri , Xbos.loc[Xbos.CRIM >= 45])
```

(1 pt.) Now try one with multiple conditions, CRIM >= 45 and AGE < 100 and assign to b

In [61]:
```
b = Xbos.loc[(Xbos["CRIM"]>=45) & (Xbos["AGE"]<100)]
b.info
```

Out[61]:
```
<bound method DataFrame.info of         CRIM    ZN  INDUS  CHAS    NOX     R
M   AGE     DIS    RAD    TAX  \
380  88.9762  0.0   18.1   0.0  0.671  6.968  91.9  1.4165  24.0  666.0

     PTRATIO      B  LSTAT
380     20.2  396.9  17.21  >
```

In [62]:
```
assert_frame_equal (b, Xbos.loc[(Xbos.CRIM >= 45) & (Xbos.AGE < 100)] )
```

You can also use loc to update the values of a particular column(s) on selected row(s), for example you have a dataframe called data and you have rows of information about individuals. You want to select all people over the age of 18, and then update the column 'age_level' with an 'A' (for adult). you would do this by

data.loc[(data.age >= 18), ['age_level']] = 'A' (Hopefully you were wondering, why are the dot method and bracket method used here? Read on for a little explanation here: https://stackoverflow.com/questions/41130255/what-is-the-difference-between-using-squared-brackets-or-dot-to-access-a-column)

(1 pt.) Take your Xbos, find all instances where AGE >80, and update the TAX column to 245.0. Check to see it worked.

In [63]:
```
#update TAX where AGE > 80
Xbos.loc[Xbos["AGE"]>80,"TAX"]=245.0
#check it worked (all rows with AGE > 80 should now have TAX == 245.00)
Xbos.loc[Xbos["AGE"]>80, ["AGE","TAX"]].head()
```

Out[63]:

	AGE	TAX
7	96.1	245.0
8	100.0	245.0
9	85.9	245.0
10	94.3	245.0
11	82.9	245.0

Show your update worked by printing out your previous query below.

In [64]:
```
print(Xbos.loc[Xbos["AGE"]>80,["AGE","TAX"]])
```

```
      AGE    TAX
7    96.1  245.0
8   100.0  245.0
9    85.9  245.0
10   94.3  245.0
11   82.9  245.0
..    ...    ...
491  98.8  245.0
492  83.5  245.0
503  91.0  245.0
504  89.3  245.0
505  80.8  245.0

[240 rows x 2 columns]
```

Other necessary pandas tasks are to drop columns, copy columns to make new dataframes, and append columns into a table. To copy a column from Xbos into its own dataframe, you can do this: newbos = Xbos.TAX.copy() Then you can drop the TAX column from Xbos: Xbos.drop(['TAX'], axis =1) and add it back again. Xbos.append(newbos) You will need to do this particularly if your datasets have the target cols included in the dataframe, or separate, just be aware of this! On this, you can merge series and dataframes, concatenate, do types of joins, and so on.

Pandas is great for dealing with missing data. You can remove rows or cols with missing data, or impute them to other values such as the mean or mode.

(1pt.) Run a query that finds NANs in the entire dataset Xbos, both rows and columns. Assign to the variable r and print results.

In [65]:
```
r=Xbos[Xbos.isna().any(axis=1)]
print(r)
```

```
Empty DataFrame
Columns: [CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LS
TAT]
Index: []
```

Let's add a row to Xbos. There are many ways to do this. Create a Pandas series and call it new_row.

In [66]:
```
new_row = pd.DataFrame({'CRIM':[2], 'ZN':[87], 'INDUS':[92], 'AGE':[97]})
```

Append row to the dataframe

In [67]:
```
Xbos = pd.concat((Xbos, new_row))
Xbos.tail()
```

Out[67]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1.0	273.0	21.0
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1.0	245.0	21.0
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1.0	245.0	21.0
505	0.04741	0.0	11.93	0.0	0.573	6.030	80.8	2.5050	1.0	245.0	21.0
0	2.00000	87.0	92.00	NaN	NaN	NaN	97.0	NaN	NaN	NaN	NaN

Let's add a column of data to Xbos, and give it a default value of 0.

In [68]:
```
Xbos['Address'] = 0
Xbos.tail()
```

Out[68]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1.0	273.0	21.0
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1.0	245.0	21.0
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1.0	245.0	21.0
505	0.04741	0.0	11.93	0.0	0.573	6.030	80.8	2.5050	1.0	245.0	21.0
0	2.00000	87.0	92.00	NaN	NaN	NaN	97.0	NaN	NaN	NaN	NaN

Visualizing and Examining data: the Iris Dataset

In this section, you will start using the scikit-learn Python package and examine a classic dataset used for machine learning. This section will give you practice in thinking about data, which is the often the most important aspect of machine learning.

Now we are going to import the Iris dataset right from sklearn. There are other versions available as well on the web-- keep that in mind, some vary slightly or are subsets of a larger set! We are going to load the dataset, and examine its properties (you may have to search these pandas commands). We will then look at some preprocessing and visualizations.

In [69]:
```
from sklearn import datasets # check these datasets out for built ins
# We are going to load the famous iris dataset.
```

In [74]:
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
import numpy as np
import pandas as pd
# import some data to play with
iris = datasets.load_iris()
#type(iris)

data1 = pd.DataFrame(data= np.c_[iris['data'], iris['target']],
                     columns= iris['feature_names'] + ['target'])
# Notice  np.c_  What is this doing? type np.c_? for help.

#np.c is a mupy concat helper. it turns 1D target into a column and then cor
```

Question 12 (6 pts)

In the cells below, look at data1. Answer each of these questions in a separate cell, using numpy and pandas functions.

Use markdown cells to answer the written questions.

1. General written question: Describe in a few sentences what this data is. You can get it from the actual set, or you can look it up online, either way is fine.
2. What is the data types?
3. What are the dimensions?
4. Is there any missing data?
5. What is the statistical overview? (mean, std etc.)
6. General written question: What is the target column? What can it be used for?

Use the code cells below to get your answers and type in your answers in the markdown cell after that.

1. The Iris dataset is a small classification dataset. It contains measurements of iris flowes - sepal length, sepal width, petal length and petal width for 150 samples. Each sample belongs to one of three iris species.

In [75]:
```
data1.dtypes
```

Out[75]:
```
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target               float64
dtype: object
```

In [76]:
```
data1.shape
```

Out[76]: (150, 5)

In [79]:
```
# missing per column
data1.isna().sum()
# see is ANY missing exists at all
data1.isna().any().any()
```

Out[79]: False

In [80]:
```
data1.describe()
```

Out[80]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

6. 'target' is the label for each flower (the species category). In scikit learn's iris dataset, it's encoded as integers 0,1,2 corresponding to the three species in iris.target_names

You *could* rename colums as data1.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width'] What might be wrong with this way of renaming the columns?

1. You would drop a column name and hence data1.columns will raise a ValueError.Length mismatch
2. It's easy to make a mistake. Setting data1.columns-... replaces all columns names at once, so a small mistake in order/length can break the dataset. It is safer to perhaps name the columns explicitly, so we only change what we intend and keep the target intact

Correlation between different features

There's lots to think about when preprocessing a dataset. We'll be going over methods in each of the labs. The first thing you might want to do is look at the correlation between two features. The Pearson correlation coefficient (named for Karl Pearson) can be used to summarize the strength of the linear relationship between two data samples.

The Pearson's correlation coefficient is calculated as the covariance of the two variables divided by the product of the standard deviation of each data sample. It is the normalization of the covariance between the two variables to give an interpretable score. In statistics, the Pearson correlation coefficient, also referred to as Pearson's r, the Pearson product-moment correlation coefficient (PPMCC), or the bivariate correlation, is a statistic that measures linear correlation between two variables X and Y. It has a value between +1 and −1. A value of +1 is total positive linear correlation, 0 is no linear correlation, and −1 is total negative linear correlation.

In our case, X and Y are two columns of our Iris dataset. While numpy, scipy, and sklearn have methods to calculate correlations of features, we are going to do this "by hand". Use only numpy! Do not use any corr() built in functions!

Bear with me, this may seem pointless or tedious but it is not! The reason is you will be calculating a formula with vectors-- this is definitely something you will see in machine learning and will get you familiar with using numpy.

to calculate r, this is the formula you will translate into numpy:

$$r = \frac{n\sum(XY) - (\sum(X)(\sum(Y)}{\sqrt{[n\sum X^2 - (\sum X)^2][n\sum Y^2 - (\sum Y)^2]}}$$

Where X and Y are feature columns (i.e. sepal width and sepal length) and n is the number of elements in each column.

Question 13 (3 pts)

Calculate the Pearson correlations for your Iris dataset in the cell below. (Many ways to do this, if you want to get fancy for practice go for it). For this correlation question, feel free to use more than one cell, just make it clear.

In [83]:
```
import numpy as np

# extract just the 4 feature columns and not the target
X= data1[iris["feature_names"]].to_numpy()
feature_names=iris["feature_names"]
```

In [84]:
```
def pearson_r(x,y):
    x=np.asarray(x)
    y=np.asarray(y)
    n=x.size
```

```
    num = n*np.sum(x*y) - (np.sum(x)*np.sum(y))
    den = np.sqrt((n*np.sum(x**2) -(np.sum(x)**2))*(n*np.sum(y**2) - (np.sum
    return num/den
```

In [85]:
```
#Correlation matrix
R = np.empty((X.shape[1],X.shape[1]))
for i in range(X.shape[1]):
    for j in range(X.shape[1]):
        R[i,j] = pearson_r(X[:,i],X[:,j])
```

In [86]:
```
print("Feature names:",feature_names)
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (c
m)', 'petal width (cm)']
```

In [87]:
```
print("Pearson correlation matrix : \n",R)
```

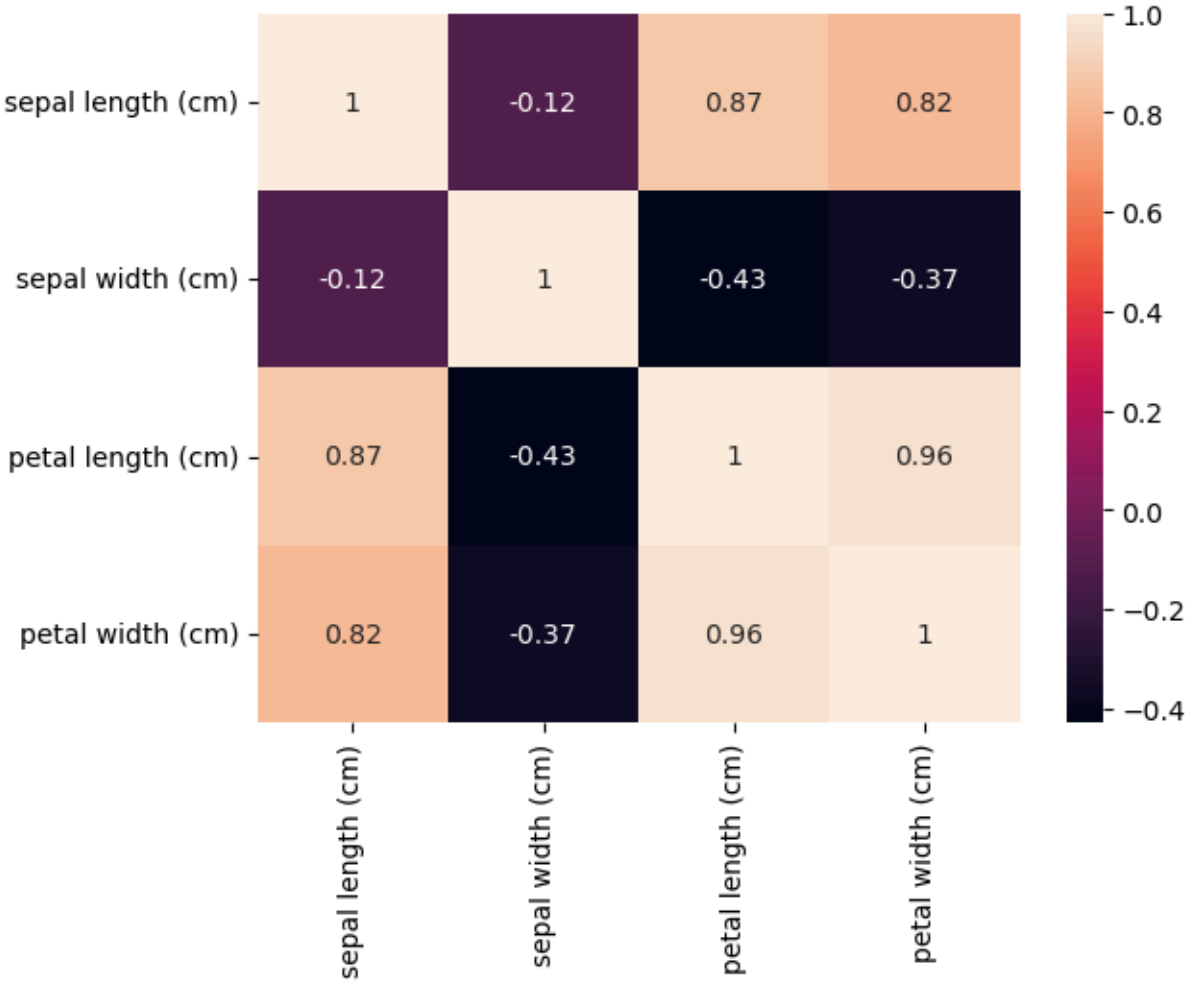
```
Pearson correlation matrix :
 [[ 1.         -0.11756978  0.87175378  0.81794113]
 [-0.11756978  1.         -0.4284401  -0.36612593]
 [ 0.87175378 -0.4284401   1.          0.96286543]
 [ 0.81794113 -0.36612593  0.96286543  1.        ]]
```

Question 14 (1 pt)

What are the two most correlated features?

To help answer this we can calculate the correlation matrix and create a map of the correlations between features.

In [88]:
```
# Here, we show you a seaborn heatmap. It is a matrix of the correlations be
# color to illustrate intensity of correlations. This will become very usefu
# are imporant for training your models.
import seaborn as sns
data2 = data1.drop('target', axis=1)
sns.heatmap(data2.corr(), annot = True);
#annot = True adds the numbers onto the squares
```



Based on the plot, the biggest off-diagonal correlation is 0.96 between "petal length" and "petal width". These are the two most correlated features !

Question 15 (2 pts)

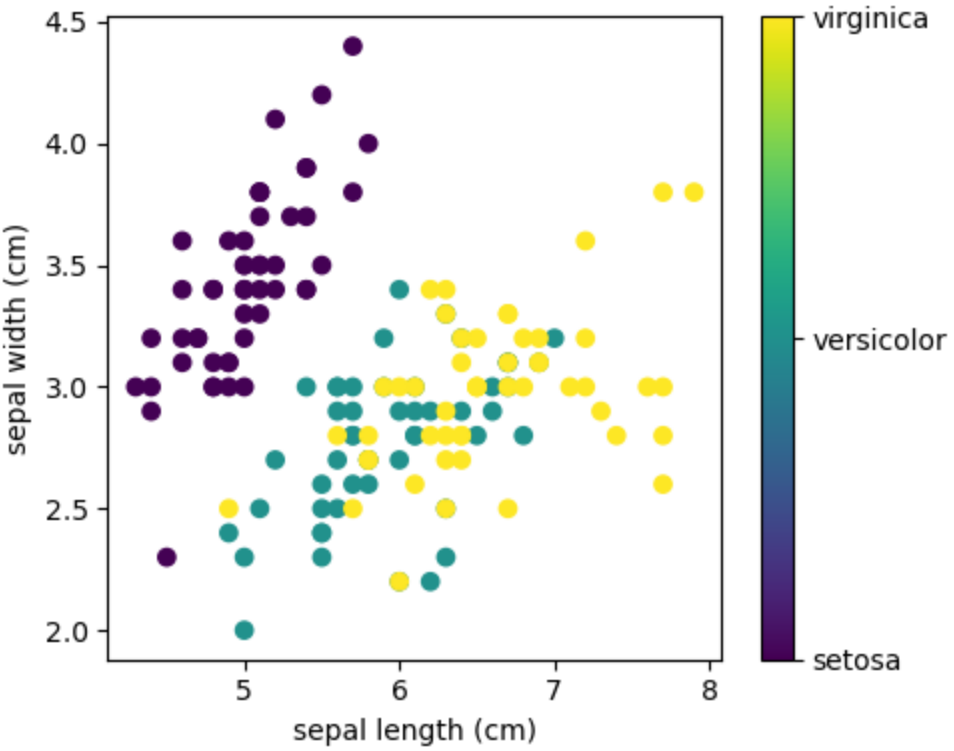
In [89]:
```
# Let's make a scatter plot of the data

# The indices of the features that we are plotting
x_index = 0
y_index = 1
# this formatter will label the colorbar with the correct target names
# notice that the target has numerical values, the target_names exist in the
# we just plot right from the dataset, not from our dataframe.
formatter = plt.FuncFormatter(lambda i, *args: iris.target_names[int(i)])

# In the cell below the plot, can you explain what this lambda function is d
# Do you understand how the data is gathered and shown?

plt.figure(figsize=(5, 4))
plt.scatter(iris.data[:, x_index], iris.data[:, y_index], c=iris.target)
plt.colorbar(ticks=[0, 1, 2], format=formatter)
plt.xlabel(iris.feature_names[x_index])
plt.ylabel(iris.feature_names[y_index])
```

```
plt.tight_layout()
plt.show()
```



(1pt.) **What is the lambda function/funcformatter doing?** Answer in the next cell.

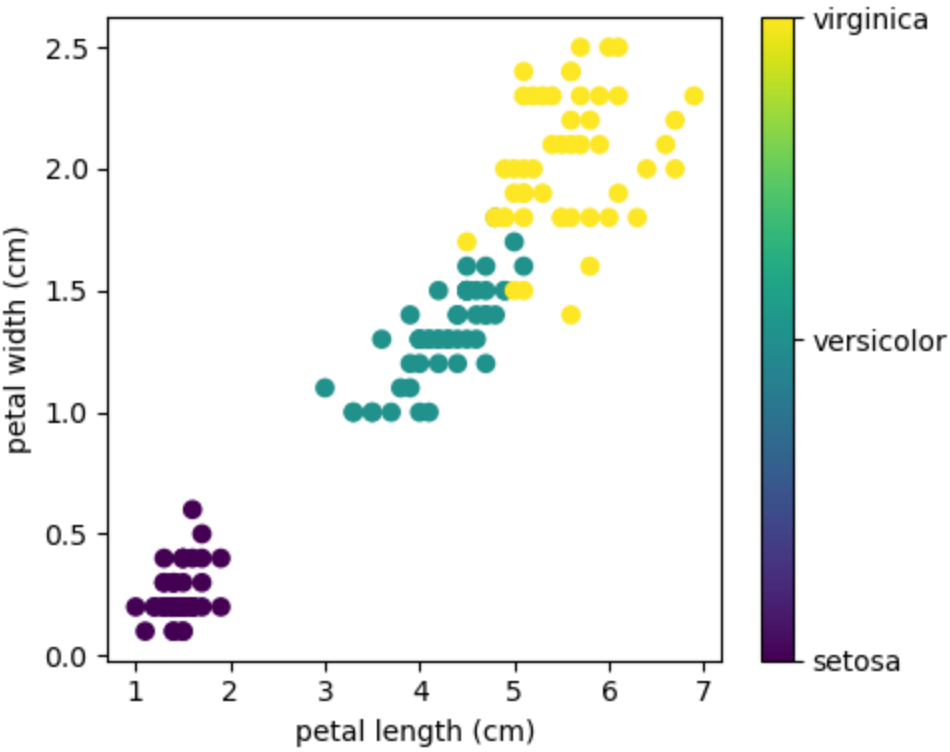
The lambda function is assigning names to 0,1,2 numbers. Thus 0 -> setosa, 1 -> versicolor and 2 -> virginica

(1pt.) Copying and pasting code, **now plot petal length vs petal width by changing the x_index and y_index values.**

In [91]:
```
x_index = 2
y_index = 3

formatter = plt.FuncFormatter(lambda i, *args: iris.target_names[int(i)])

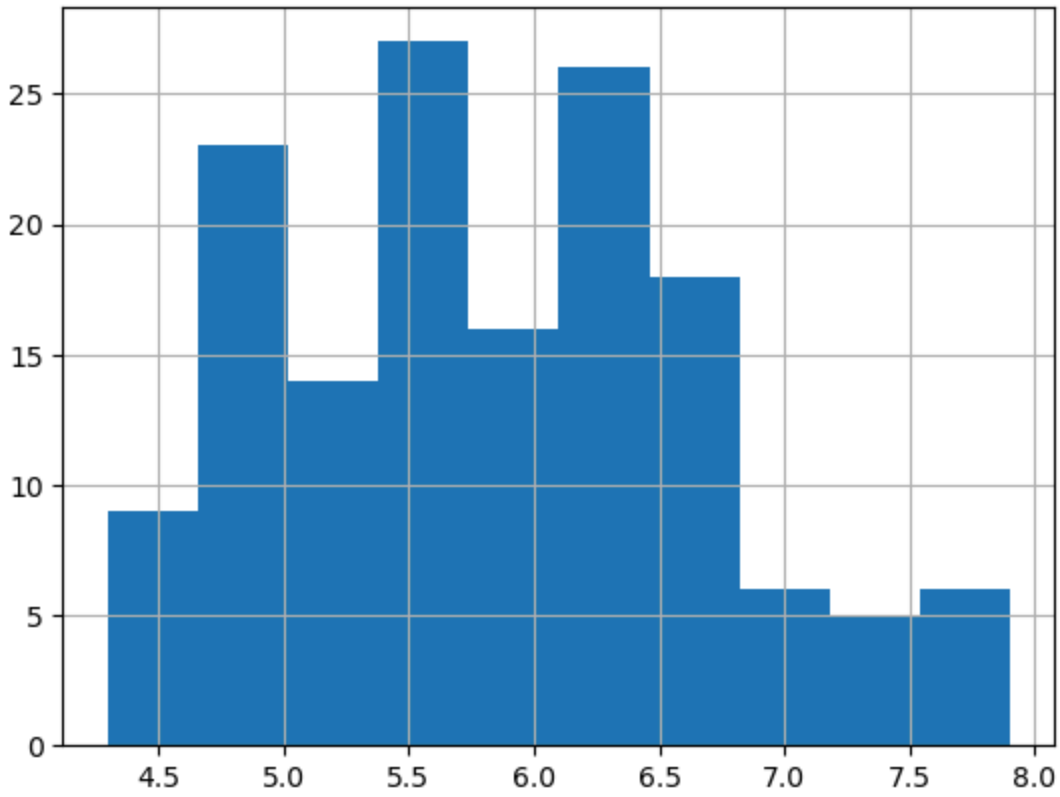

plt.figure(figsize=(5, 4))
plt.scatter(iris.data[:, x_index], iris.data[:, y_index], c=iris.target)
plt.colorbar(ticks=[0, 1, 2], format=formatter)
plt.xlabel(iris.feature_names[x_index])
plt.ylabel(iris.feature_names[y_index])
plt.tight_layout()
plt.show()
```



Pandas built-in Viz Pandas has built in data visualization, based on matplotlib, so we show a few examples. (Feel free to use any plotting package you like for this course, but these are pretty convenient if you have a pandas dataframe ready to go.

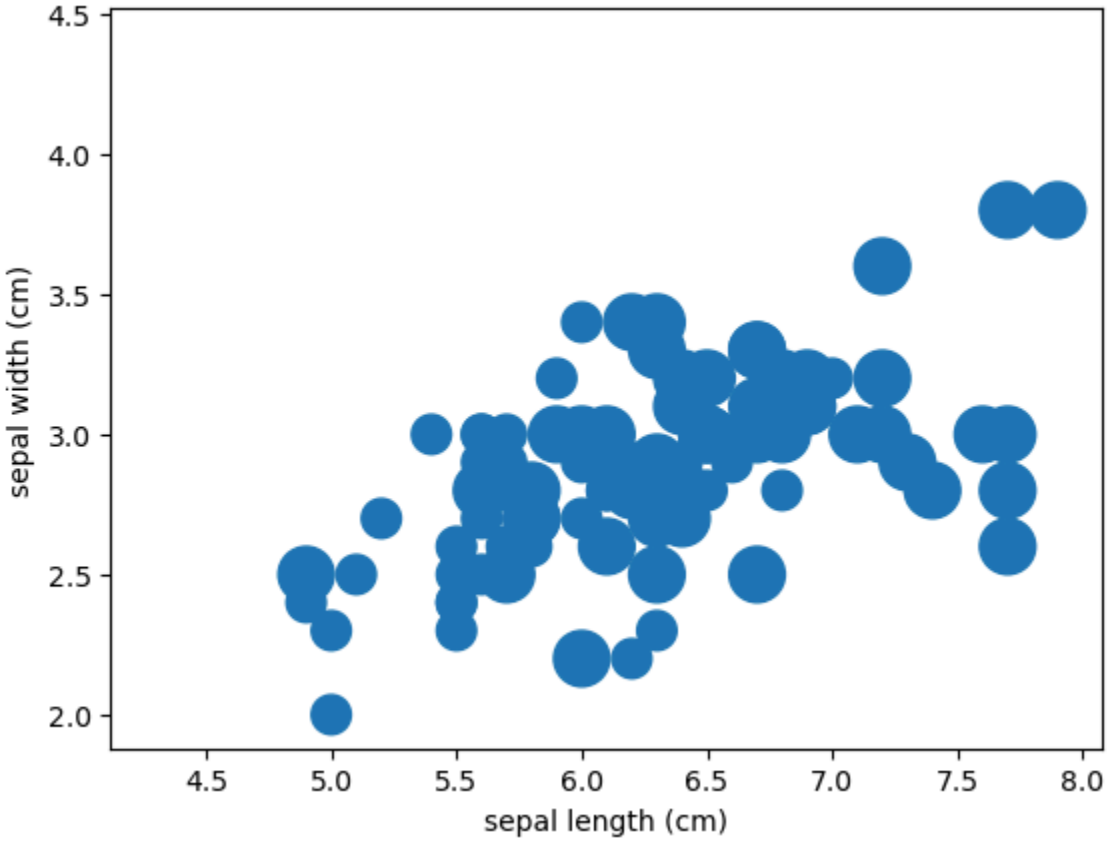
In [92]:
```
data1['sepal length (cm)'].hist()
```

Out[92]:
```
<Axes: >
```



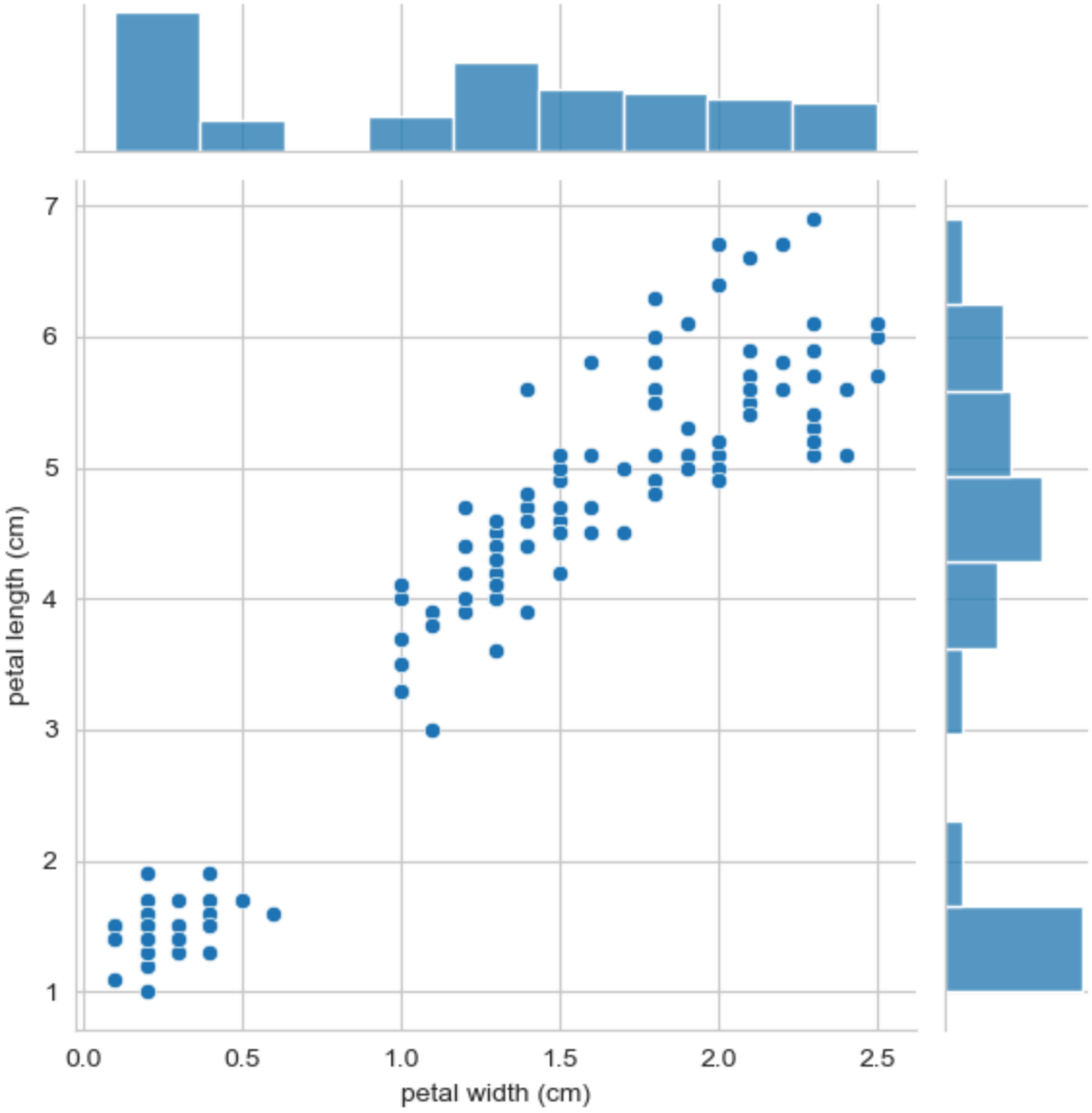
In [93]:
```
data1.plot.scatter(x='sepal length (cm)', y= 'sepal width (cm)', s=data1['ta
```

Out[93]: <Axes: xlabel='sepal length (cm)', ylabel='sepal width (cm)'>



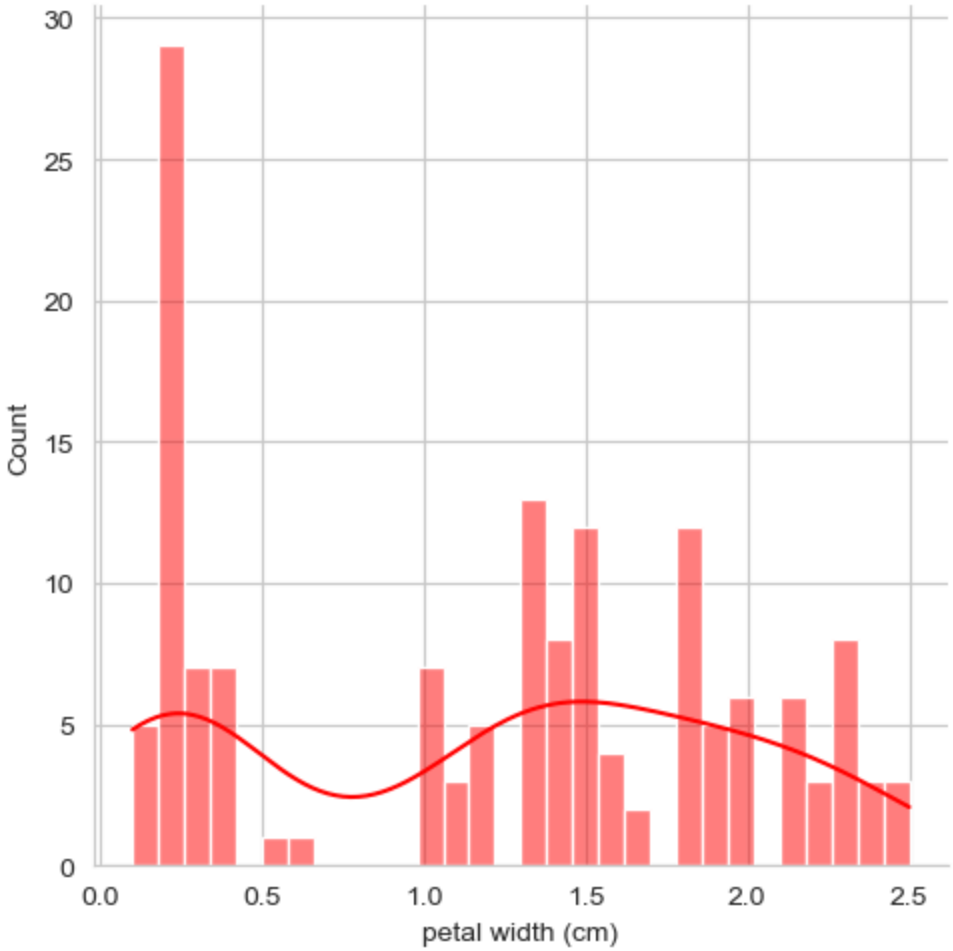
In [94]:
```
# Let's do some plotting with seaborn.
# We start with a jointplot
sns.set_style('whitegrid')
sns.jointplot(x= 'petal width (cm)', y = 'petal length (cm)', data=data1)
```

Out[94]: <seaborn.axisgrid.JointGrid at 0x13833f6d0>



In [95]:
```
# (1pt.) Using data1 and petal width, create a seaborn displot with 30 bins,

### BEGIN SOLUTION
sns.displot(data=data1, x='petal width (cm)', bins=30, kde=True, color='red'
plt.show()
### END SOLUTION
```



Plotting Data

For this final section of the lab, we are going to be plotting using seaborn and the Titanic dataset. Your tasks will be to plot various directives. (There are also lots of Jupyter notebooks on Kaggle.com from challenges using the Titanic dataset, FYI)

In [96]:
```
titanic = sns.load_dataset('titanic') #notice where we are getting Titanic f
titanic.head()
```

Out[96]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	ad
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	

Question 16 (16 pts)

For this question you are going to make a series of plots with seaborn in the cells below using the Titanic dataset. **Remember to label your axes!** Include units if you have them.

Part 1: In a cell below, create a correlation matrix of features (you can use any method you like)

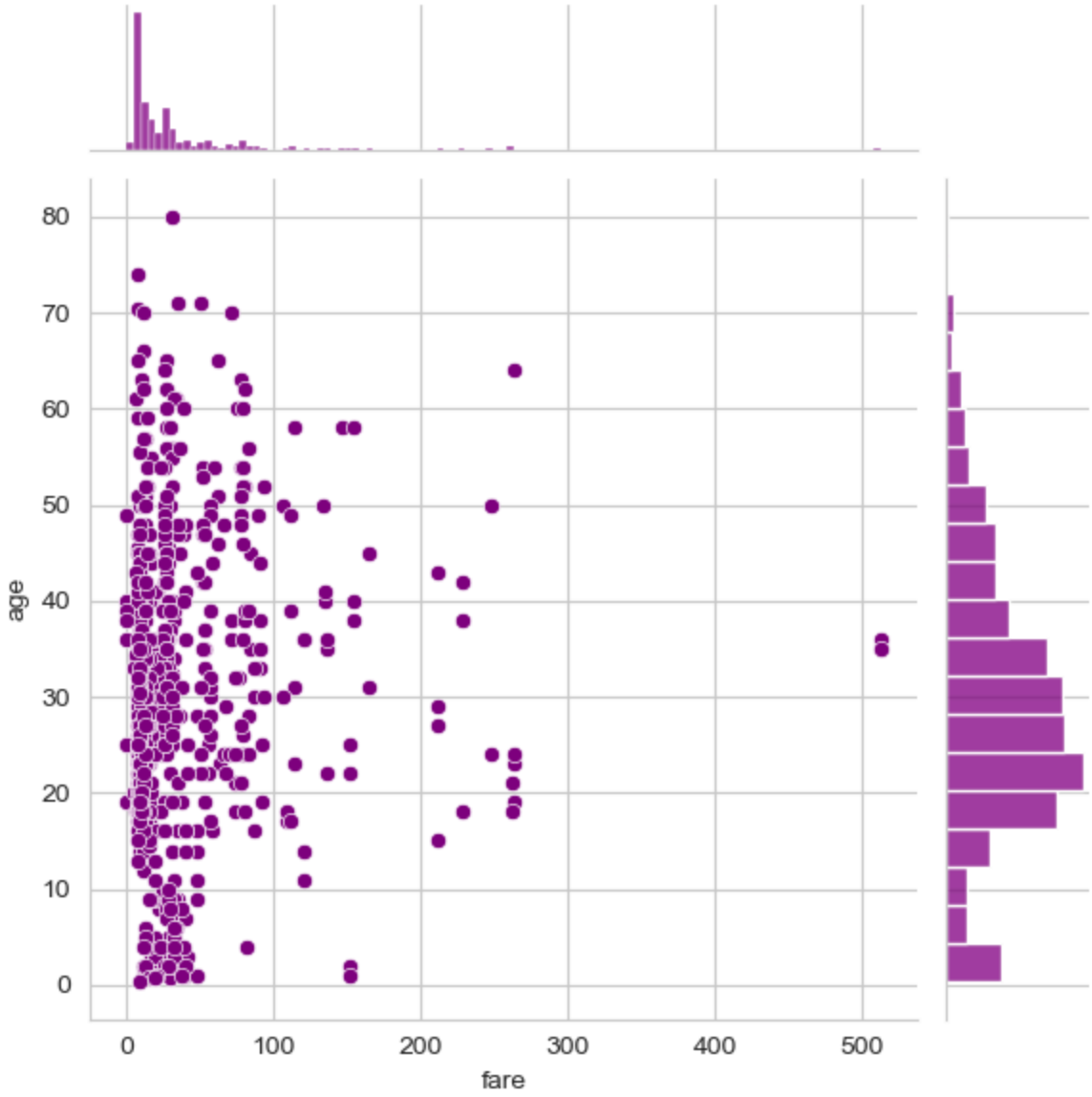
In [99]:
```
corr = titanic.select_dtypes(include="number").corr()
corr
```

Out[99]:

	survived	pclass	age	sibsp	parch	fare
survived	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
pclass	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
age	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
sibsp	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
parch	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
fare	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

Part 2: Using seaborn and Titanic, in a cell below create a joint plot with the x axis as fare, the y axis as age. feel free to experiment with size and/or colors if you like.

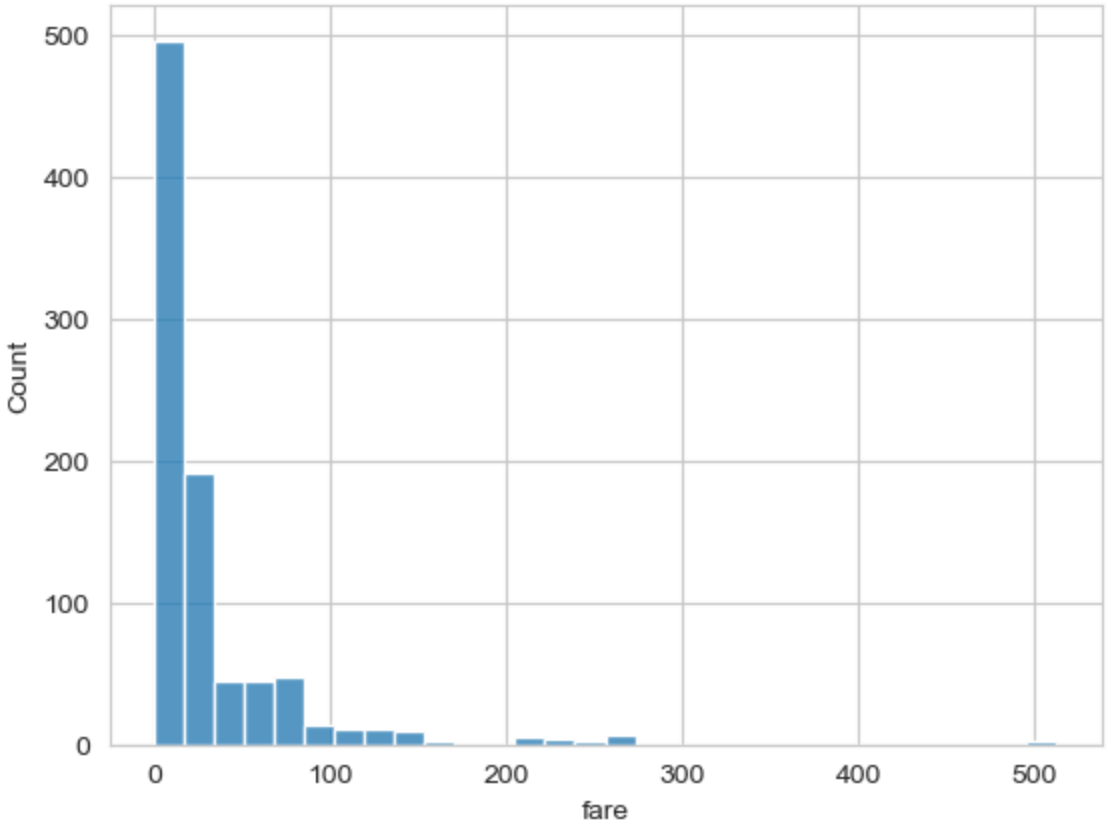
In [100…
```
sns.jointplot(data=titanic, x="fare", y="age", height=6, color="purple")
plt.show()
```



Part 3: In a cell below, create a histogram of the fare, kde= False, and choose an appropriate number of bins

In [102…

```
sns.histplot(data=titanic,x="fare",bins=30,kde=False)
plt.show()
```



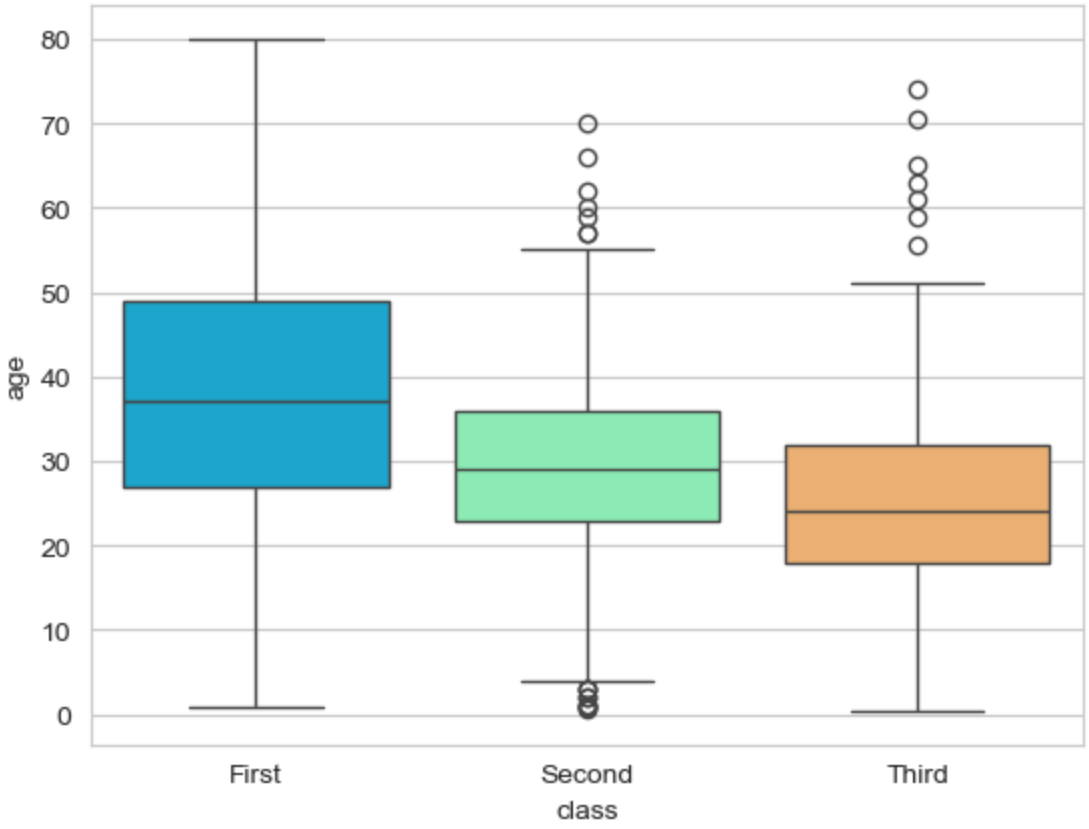
Part 4: Create a box plot with x axis being class type, and y axis age. Try palette ='rainbow'

In [104…
```
sns.boxplot(data=titanic, x="class",y="age",palette="rainbow")
plt.show()
```

```
/var/folders/ww/jyhpvg8x3md06rt6f3f93jw00000gn/T/ipykernel_59578/146113631.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=titanic, x="class",y="age",palette="rainbow")
```

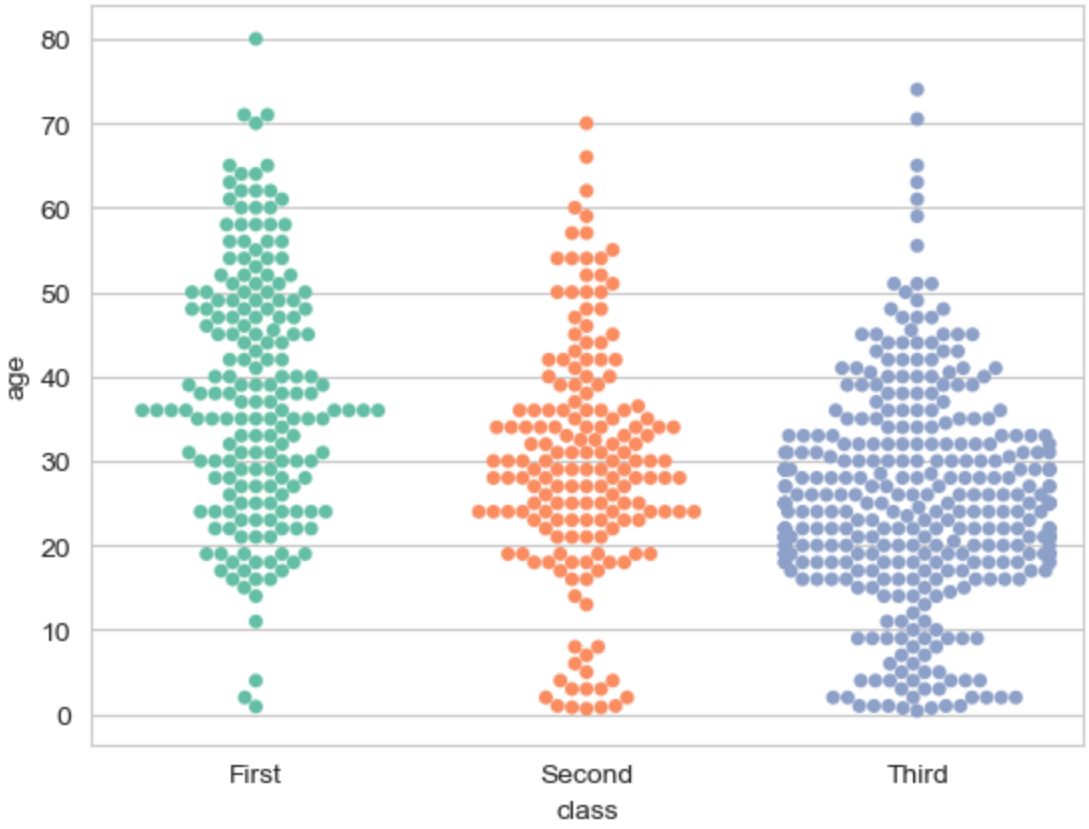


Part 5: Create a swarm plot with x axis class, and y axis age, try the palette 'Set2'. You might get a warning-- can you fix it? (it is okay to leave the warning, however)

In [105…

```
sns.swarmplot(
    data=titanic,
    x="class", y="age",
    hue="class", palette="Set2",
    dodge=False, legend=False
)
plt.show()
```

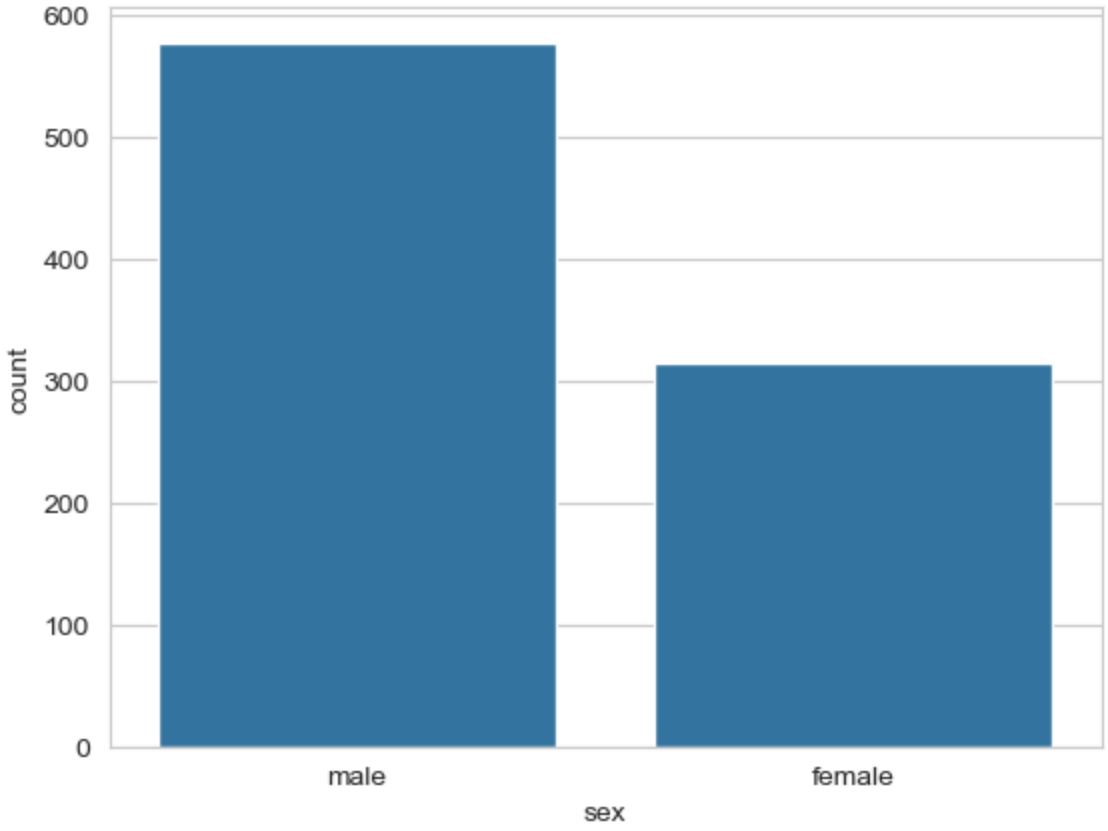
```
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/seaborn/categorical.py:3399: UserWarning: 15.2% of the points cannot be placed; you may want to decrease the size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```



Part 6: Create a count plot with x axis using sex.

In [109…

```
sns.countplot(data=titanic, x="sex")
plt.show()
```



Part 7: Create a heatmap of the correlations. Title it "Titanic"

In [110…

```
corr=titanic.select_dtypes(include="number").corr()

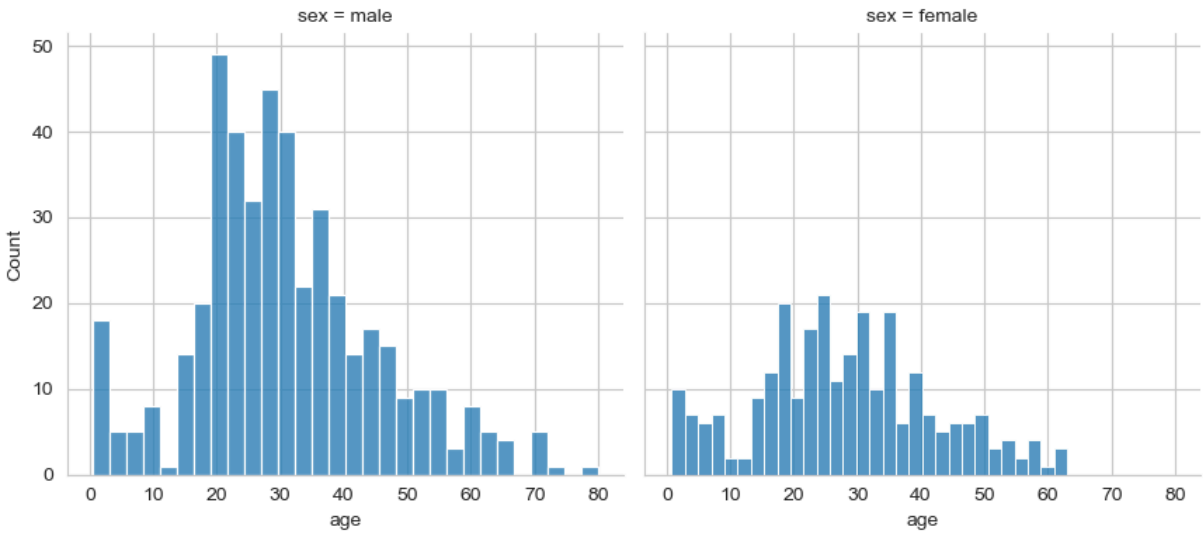
plt.figure(figsize=(8,6))
sns.heatmap(corr,annot=True,cmap="coolwarm",square=True)
plt.title("Titanic")
plt.tight_layout()
plt.show()
```



Part 8: Create a facet grid using sex as the columns, then map the histogram on age.

In [113…

```
g = sns.FacetGrid(titanic, col="sex", height=4, aspect=1.1)
g.map(sns.histplot, "age", bins=30)
plt.show()
```



In [114…

```
# answer people are missing
# g = sns.FacetGrid(data=titanic,col='sex')
# g.map(plt.hist,'age')
```

Good Job!

Rerun your cells and submit your assignment.